

# MidnightJS SDK Reference

---

*Privacy-Preserving DApp Development Framework for Midnight*

---

**Generated:** 2026-03-13

**Evidence tests:** 249

**SDK version:** 3.2.0-rc.3

# Table of Contents

---

## 1. Introduction

What is MidnightJS?

The provider pattern

The transaction flow

Mental model

Three oracles for verification

How to read this document

Methodology

## 2. API Surface

Summary

contracts @midnight-ntwrk/midnight-js-contracts

fetch-zk-config-provider @midnight-ntwrk/midnight-js-fetch-zk-config-provider

http-client-proof-provider @midnight-ntwrk/midnight-js-http-client-proof-provider

indexer-public-data-provider @midnight-ntwrk/midnight-js-indexer-public-data-provider

level-private-state-provider @midnight-ntwrk/midnight-js-level-private-state-provider

logger-provider @midnight-ntwrk/midnight-js-logger-provider

network-id @midnight-ntwrk/midnight-js-network-id

node-zk-config-provider @midnight-ntwrk/midnight-js-node-zk-config-provider

types @midnight-ntwrk/midnight-js-types

utils @midnight-ntwrk/midnight-js-utils

TypeScript Type Verification

## 3. Provider Interfaces

3.1 PrivateStateProvider

3.2 PublicDataProvider

3.3 ZKConfigProvider

3.4 ProofProvider

3.5 WalletProvider

3.6 MidnightProvider

3.7 LoggerProvider

## 4. Contract Lifecycle

4.1 Deploying a Contract

4.2 Finding a Deployed Contract

4.3 Calling a Circuit

4.4 Querying State

4.5 Transaction Submission

## 5. Transaction Model

5.1 Transaction Context and Scoping

5.2 Transaction Interfaces

5.3 Transaction Data Types

## 6. Error Handling

## 7. Contract Runtime Helpers

## 8. Network Configuration

## 9. Utilities

## 10. Full-Flow Integration Tests

## 11. End-to-End Examples

Example 1: Counter Contract — Deploy, Call, Observe

Example 2: Private Counter — Witnesses and Authorization

Example 3: State Observation — Queries, Verification, and Observables

Example 4: Low-Level Transaction Pipeline

Example 5: Contract Maintenance — Authority and Verifier Keys

Example 6: Private State — Encryption, Backup, and Migration

Example 7: Utility Functions, Type Helpers, and Network Configuration

Example 8: Advanced Deploy Pipeline and Provider Configuration

## Appendix A: Execution Oracle Tool

Input

Pipeline

Tamper modes

## Appendix B: API Surface Extraction Tool

What it extracts

Modes

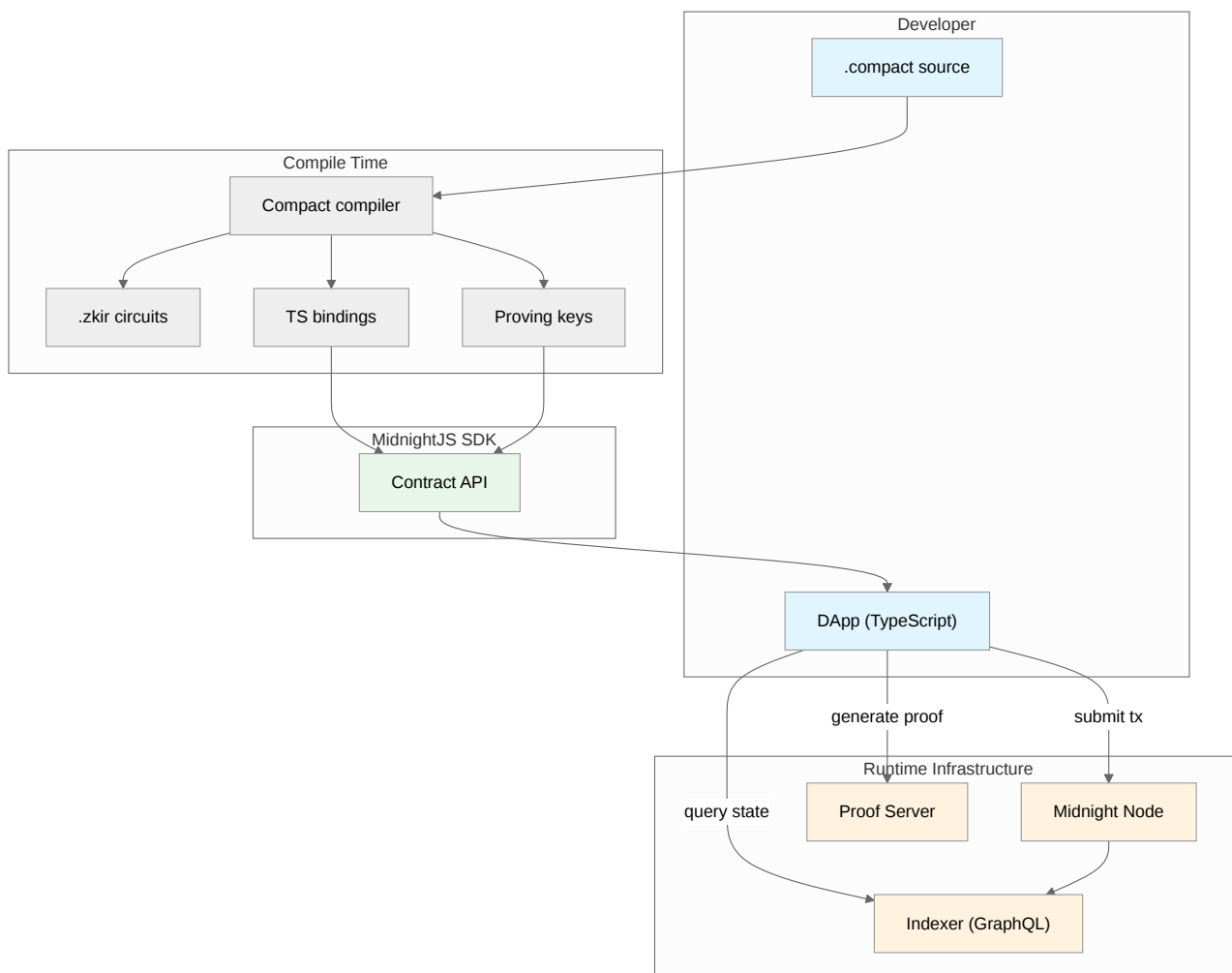
## Appendix C: Coverage Checker Tool

## Appendix D: Reproducing This Document

# 1. Introduction

## What is MidnightJS?

High-level explanation: MidnightJS is the TypeScript SDK for building privacy-preserving DApps on the Midnight blockchain. It provides the complete lifecycle for contract interaction: deploying contracts, calling circuits, generating zero-knowledge proofs, and submitting transactions -- all through a modular provider pattern.



## The provider pattern

MidnightJS uses **pluggable providers** -- each capability (storage, networking, proving, etc.) is a separate interface that can be swapped independently. A DApp assembles the providers it needs:

```
const providers: MidnightProviders = {
  privateStateProvider, // Local encrypted state
  publicDataProvider,   // Blockchain queries (GraphQL)
  zkConfigProvider,     // ZK artifact retrieval
  proofProvider,        // Proof generation
  walletProvider,        // Tx balancing & signing
  midnightProvider,     // Tx submission
  loggerProvider,       // Optional diagnostics
};
```

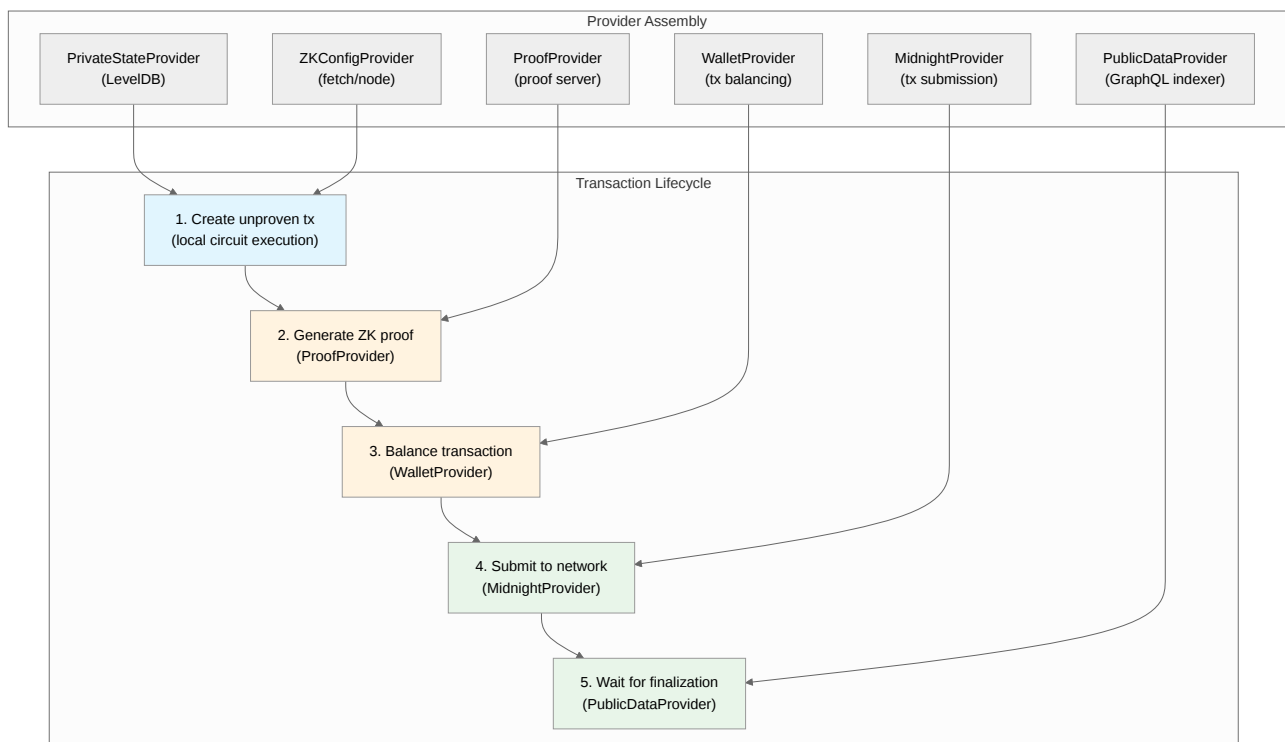
## The transaction flow

Every contract interaction follows a five-stage pipeline:

1. Create unproven transaction (local circuit execution)
2. Generate proofs via ProofProvider
3. Balance transaction via WalletProvider
4. Submit via MidnightProvider
5. Wait for finalization via PublicDataProvider

## Mental model

A MidnightJS DApp assembles providers and uses them to interact with contracts through a five-stage transaction lifecycle:



The SDK offers three abstraction levels for this lifecycle:

- **High-level:** `deployContract` / `submitCallTx` — handles all five stages automatically
- **Mid-level:** `createUnprovenCallTx` + `submitTx` — split creation from submission for inspection

- **Low-level:** `createUnprovenCallTxFromInitialStates` + `submitTxAsync` — full control over state and async submission

## Three oracles for verification

This document uses three deterministic oracles:

**Oracle 1: TypeScript compiler.** Does the SDK compile? Do typed examples type-check? ( `tsc --noEmit` )

**Oracle 2: Execution oracle.** Given a compiled `.compact` contract, can we execute circuits through the SDK runtime and verify the resulting proof data via the ZKIR constraint checker?

**Oracle 3: Proof server** (stretch goal). Does the full prove/balance/submit pipeline work against a dockerized proof server?

## How to read this document

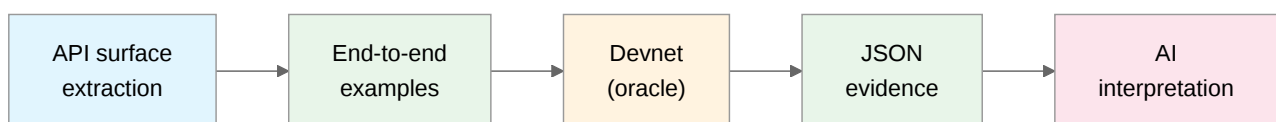
This document follows the **Aletheia principle**: every claim an AI makes must be backed by deterministically produced evidence.

- `[deterministic]` -- produced by automated tools, no AI
- `[AI-generated]` -- prose written by an AI to interpret evidence

## Methodology `[AI-generated]`

This document applies the Aletheia principle to SDK documentation through worked examples verified against a local devnet. The approach mirrors the scientific method:

1. **Definition.** The API surface is extracted mechanically from the TypeScript source — every exported symbol, its type, and its package.
2. **Experiment.** End-to-end examples exercise the SDK against a local devnet (node + indexer + proof server). Each example is a runnable script that deploys contracts, calls circuits, and observes state.
3. **Evidence.** Each example produces a JSON evidence file recording every step: what was called, what was returned, whether claims hold.
4. **Interpretation.** AI-generated prose explains the evidence and connects it to the SDK's design patterns.



The devnet is the oracle. Every claim about SDK behavior — "deploying returns a contract address," "calling increment changes the counter" — is backed by an actual execution against a real (local) blockchain. The evidence files are the experimental results.

## 2. API Surface [deterministic]

Source: `upstream/midnight-js/packages/`, extracted by `tools/extract-api-surface.mjs`

The complete public API surface of the SDK -- every exported symbol across all packages. This section is auto-generated from the TypeScript barrel exports.

Extracted from `upstream/midnight-js/packages/` Generated: 2026-03-13T21:26:31.178Z

### Summary

| Package                                   | Exports    | Version |
|-------------------------------------------|------------|---------|
| <code>contracts</code>                    | 91         | 3.2.0   |
| <code>fetch-zk-config-provider</code>     | 1          | 3.2.0   |
| <code>http-client-proof-provider</code>   | 5          | 3.2.0   |
| <code>indexer-public-data-provider</code> | 5          | 3.2.0   |
| <code>level-private-state-provider</code> | 10         | 3.2.0   |
| <code>logger-provider</code>              | 1          | 3.2.0   |
| <code>network-id</code>                   | 3          | 3.2.0   |
| <code>node-zk-config-provider</code>      | 1          | 3.2.0   |
| <code>types</code>                        | 67         | 3.2.0   |
| <code>utils</code>                        | 12         | 3.2.0   |
| <b>Total</b>                              | <b>196</b> |         |

**contracts** (@midnight-ntwrk/midnight-js-contracts)

| Export                                                 | Kind      | Source               |
|--------------------------------------------------------|-----------|----------------------|
| CallOptions                                            | type      | ./call               |
| CallOptionsBase                                        | type      | ./call               |
| CallOptionsProviderDataDependencies                    | type      | ./call               |
| CallOptionsWithArguments                               | type      | ./call               |
| CallOptionsWithPrivateState                            | type      | ./call               |
| CallOptionsWithProviderDataDependencies                | type      | ./call               |
| CallResult                                             | type      | ./call               |
| CallResultPrivate                                      | type      | ./call               |
| CallResultPublic                                       | type      | ./call               |
| CallTxFailedError                                      | class     | ./errors             |
| CallTxOptions                                          | type      | ./unproven-call-tx   |
| CallTxOptionsBase                                      | type      | ./unproven-call-tx   |
| CallTxOptionsWithPrivateStateId                        | type      | ./unproven-call-tx   |
| CircuitCallTxInterface                                 | type      | ./tx-interfaces      |
| CircuitMaintenanceTxInterface                          | type      | ./tx-interfaces      |
| CircuitMaintenanceTxInterfaces                         | type      | ./tx-interfaces      |
| ContractConstructorOptions                             | type      | ./call-constructor   |
| ContractConstructorOptionsBase                         | type      | ./call-constructor   |
| ContractConstructorOptionsProviderDataDependencies     | type      | ./call-constructor   |
| ContractConstructorOptionsWithArguments                | type      | ./call-constructor   |
| ContractConstructorOptionsWithPrivateState             | type      | ./call-constructor   |
| ContractConstructorOptionsWithProviderDataDependencies | type      | ./call-constructor   |
| ContractConstructorResult                              | type      | ./call-constructor   |
| ContractMaintenanceTxInterface                         | interface | ./tx-interfaces      |
| ContractProviders                                      | type      | ./contract-providers |
| ContractStates                                         | type      | ./get-states         |
| ContractTypeError                                      | class     | ./errors             |
| createCallTxOptions                                    | const     | ./tx-interfaces      |
| createCircuitCallTxInterface                           | const     | ./tx-interfaces      |
| createCircuitMaintenanceTxInterface                    | const     | ./tx-interfaces      |
| createCircuitMaintenanceTxInterfaces                   | const     | ./tx-interfaces      |
| createContractMaintenanceTxInterface                   | const     | ./tx-interfaces      |



| Export                                                       | Kind     | Source                                 |
|--------------------------------------------------------------|----------|----------------------------------------|
| <code>createUnprovenCallTx</code>                            | function | <code>./unproven-call-tx</code>        |
| <code>createUnprovenCallTxFromInitialStates</code>           | function | <code>./unproven-call-tx</code>        |
| <code>createUnprovenDeployTx</code>                          | function | <code>./unproven-deploy-tx</code>      |
| <code>createUnprovenDeployTxFromVerifierKeys</code>          | function | <code>./unproven-deploy-tx</code>      |
| <code>deployContract</code>                                  | function | <code>./deploy-contract</code>         |
| <code>DeployContractOptions</code>                           | type     | <code>./deploy-contract</code>         |
| <code>DeployContractOptionsBase</code>                       | type     | <code>./deploy-contract</code>         |
| <code>DeployContractOptionsWithPrivateState</code>           | type     | <code>./deploy-contract</code>         |
| <code>DeployedContract</code>                                | type     | <code>./deploy-contract</code>         |
| <code>DeployTxFailedError</code>                             | class    | <code>./errors</code>                  |
| <code>DeployTxOptions</code>                                 | type     | <code>./submit-deploy-tx</code>        |
| <code>DeployTxOptionsBase</code>                             | type     | <code>./unproven-deploy-tx</code>      |
| <code>DeployTxOptionsWithPrivateState</code>                 | type     | <code>./unproven-deploy-tx</code>      |
| <code>DeployTxOptionsWithPrivateStateId</code>               | type     | <code>./unproven-deploy-tx</code>      |
| <code>FinalizedCallTxData</code>                             | type     | <code>./tx-model</code>                |
| <code>FinalizedDeployTxData</code>                           | type     | <code>./tx-model</code>                |
| <code>FinalizedDeployTxDataBase</code>                       | type     | <code>./tx-model</code>                |
| <code>findDeployedContract</code>                            | function | <code>./find-deployed-contract</code>  |
| <code>FindDeployedContractOptions</code>                     | type     | <code>./find-deployed-contract</code>  |
| <code>FindDeployedContractOptionsBase</code>                 | type     | <code>./find-deployed-contract</code>  |
| <code>FindDeployedContractOptionsExistingPrivateState</code> | type     | <code>./find-deployed-contract</code>  |
| <code>FindDeployedContractOptionsStorePrivateState</code>    | type     | <code>./find-deployed-contract</code>  |
| <code>FoundContract</code>                                   | type     | <code>./find-deployed-contract</code>  |
| <code>getPublicStates</code>                                 | const    | <code>./get-states</code>              |
| <code>getStates</code>                                       | const    | <code>./get-states</code>              |
| <code>getUnshieldedBalances</code>                           | const    | <code>./get-unshielded-balances</code> |
| <code>IncompleteCallTxPrivateStateConfig</code>              | class    | <code>./errors</code>                  |
| <code>IncompleteFindContractPrivateStateConfig</code>        | class    | <code>./errors</code>                  |
| <code>InsertVerifierKeyTxFailedError</code>                  | class    | <code>./errors</code>                  |
| <code>PublicContractStates</code>                            | type     | <code>./get-states</code>              |
| <code>RemoveVerifierKeyTxFailedError</code>                  | class    | <code>./errors</code>                  |
| <code>ReplaceMaintenanceAuthorityTxFailedError</code>        | class    | <code>./errors</code>                  |
| <code>ScopedTransactionOptions</code>                        | type     | <code>./transaction</code>             |

| Export                                               | Kind      | Source                                     |
|------------------------------------------------------|-----------|--------------------------------------------|
| <code>submitCallTx</code>                            | function  | <code>./submit-call-tx</code>              |
| <code>submitCallTxAsync</code>                       | function  | <code>./submit-call-tx</code>              |
| <code>submitDeployTx</code>                          | function  | <code>./submit-deploy-tx</code>            |
| <code>submitInsertVerifierKeyTx</code>               | const     | <code>./submit-insert-vk-tx</code>         |
| <code>submitRemoveVerifierKeyTx</code>               | const     | <code>./submit-remove-vk-tx</code>         |
| <code>submitReplaceAuthorityTx</code>                | const     | <code>./submit-replace-authority-tx</code> |
| <code>SubmittedCallTx</code>                         | type      | <code>./tx-model</code>                    |
| <code>submitTx</code>                                | const     | <code>./submit-tx</code>                   |
| <code>submitTxAsync</code>                           | const     | <code>./submit-tx</code>                   |
| <code>SubmitTxOptions</code>                         | type      | <code>./submit-tx</code>                   |
| <code>SubmitTxProviders</code>                       | type      | <code>./submit-tx</code>                   |
| <code>TransactionContext</code>                      | interface | <code>./transaction</code>                 |
| <code>TxFailedError</code>                           | class     | <code>./errors</code>                      |
| <code>UnprovenCallTxProvidersBase</code>             | type      | <code>./unproven-call-tx</code>            |
| <code>UnprovenCallTxProvidersWithPrivateState</code> | type      | <code>./unproven-call-tx</code>            |
| <code>UnprovenDeployTxOptions</code>                 | type      | <code>./unproven-deploy-tx</code>          |
| <code>UnprovenDeployTxProviders</code>               | type      | <code>./unproven-deploy-tx</code>          |
| <code>UnsubmittedCallTxData</code>                   | type      | <code>./tx-model</code>                    |
| <code>UnsubmittedDeployTxData</code>                 | type      | <code>./tx-model</code>                    |
| <code>UnsubmittedDeployTxDataBase</code>             | type      | <code>./tx-model</code>                    |
| <code>UnsubmittedDeployTxPrivateData</code>          | type      | <code>./tx-model</code>                    |
| <code>UnsubmittedDeployTxPublicData</code>           | type      | <code>./tx-model</code>                    |
| <code>UnsubmittedTxData</code>                       | type      | <code>./tx-model</code>                    |
| <code>verifierKeysEqual</code>                       | const     | <code>./find-deployed-contract</code>      |
| <code>verifyContractState</code>                     | const     | <code>./find-deployed-contract</code>      |
| <code>withContractScopedTransaction</code>           | const     | <code>./transaction</code>                 |

### `fetch-zk-config-provider` (@midnight-ntwrk/midnight-js-fetch-zk-config-provider)

| Export                             | Kind  | Source                                  |
|------------------------------------|-------|-----------------------------------------|
| <code>FetchZkConfigProvider</code> | class | <code>./fetch-zk-config-provider</code> |

**http-client-proof-provider** (@midnight-ntwrk/midnight-js-http-client-proof-provider)

| Export                    | Kind  | Source                         |
|---------------------------|-------|--------------------------------|
| DEFAULT_CONFIG            | const | ./http-client-proof-provider   |
| DEFAULT_TIMEOUT           | const | ./http-client-proving-provider |
| httpClientProofProvider   | const | ./http-client-proof-provider   |
| httpClientProvingProvider | const | ./http-client-proving-provider |
| ProvingProviderConfig     | type  | ./http-client-proving-provider |

**indexer-public-data-provider** (@midnight-ntwrk/midnight-js-indexer-public-data-provider)

| Export                    | Kind  | Source                         |
|---------------------------|-------|--------------------------------|
| IndexerFormattedError     | class | ./errors                       |
| indexerPublicDataProvider | const | ./indexer-public-data-provider |
| IndexerUtxo               | type  | ./indexer-public-data-provider |
| toUnshieldedBalances      | const | ./indexer-public-data-provider |
| toUnshieldedUtxos         | const | ./indexer-public-data-provider |

**level-private-state-provider** (@midnight-ntwrk/midnight-js-level-private-state-provider)

| Export                          | Kind  | Source                         |
|---------------------------------|-------|--------------------------------|
| decryptValue                    | const | ./storage-encryption           |
| DEFAULT_CONFIG                  | const | ./level-private-state-provider |
| levelPrivateStateProvider       | const | ./level-private-state-provider |
| LevelPrivateStateProviderConfig | type  | ./level-private-state-provider |
| migrateToAccountScoped          | const | ./level-private-state-provider |
| MigrationResult                 | type  | ./level-private-state-provider |
| PasswordRotationOptions         | type  | ./level-private-state-provider |
| PasswordRotationResult          | type  | ./level-private-state-provider |
| PrivateStoragePasswordProvider  | type  | ./storage-encryption           |
| StorageEncryption               | class | ./storage-encryption           |

**logger-provider** (@midnight-ntwrk/midnight-js-logger-provider)

| Export         | Kind  | Source            |
|----------------|-------|-------------------|
| LoggerProvider | class | ./logger-provider |

**network-id** (@midnight-ntwrk/midnight-js-network-id)

| Export       | Kind  | Source       |
|--------------|-------|--------------|
| getNetworkId | const | —            |
| NetworkId    | type  | ./network-id |
| setNetworkId | const | —            |

**node-zk-config-provider** (@midnight-ntwrk/midnight-js-node-zk-config-provider)

| Export               | Kind  | Source                    |
|----------------------|-------|---------------------------|
| NodeZkConfigProvider | class | ./node-zk-config-provider |

**types** (@midnight-ntwrk/midnight-js-types)

| Export                           | Kind      | Source                   |
|----------------------------------|-----------|--------------------------|
| All                              | type      | ./public-data-provider   |
| AnyPrivateState                  | type      | ./midnight-types         |
| AnyProvableCircuitId             | type      | ./midnight-types         |
| asContractAddress                | const     | ./contract               |
| asEffectOption                   | const     | ./contract               |
| BlockHash                        | type      | ./midnight-types         |
| BlockHashConfig                  | type      | ./public-data-provider   |
| BlockHeightConfig                | type      | ./public-data-provider   |
| ContractExecutableRuntimeOptions | type      | ./contract               |
| ContractStateObservableConfig    | type      | ./public-data-provider   |
| createProverKey                  | const     | ./midnight-types         |
| createVerifierKey                | const     | ./midnight-types         |
| createZKIR                       | const     | ./midnight-types         |
| exitResultOrError                | const     | ./contract               |
| ExportDecryptionError            | class     | ./errors                 |
| ExportPrivateStatesOptions       | interface | ./private-state-provider |
| ExportSigningKeysOptions         | interface | ./private-state-provider |
| FailEntirely                     | const     | ./midnight-types         |
| FailFallible                     | const     | ./midnight-types         |
| Fees                             | type      | ./midnight-types         |
| FinalizedTxData                  | interface | ./midnight-types         |
| ImportConflictError              | class     | ./errors                 |
| ImportPrivateStatesOptions       | interface | ./private-state-provider |
| ImportPrivateStatesResult        | interface | ./private-state-provider |
| ImportSigningKeysOptions         | interface | ./private-state-provider |
| ImportSigningKeysResult          | interface | ./private-state-provider |
| InvalidExportFormatError         | class     | ./errors                 |
| InvalidProtocolSchemeError       | class     | ./errors                 |
| KeyMaterialProvider              | type      | ./zk-config-provider     |
| Latest                           | type      | ./public-data-provider   |
| LoggerProvider                   | interface | ./logger-provider        |
| LogLevel                         | enum      | ./logger-provider        |

| Export                                     | Kind              | Source                                 |
|--------------------------------------------|-------------------|----------------------------------------|
| <code>makeContractExecutableRuntime</code> | const             | <code>./contract</code>                |
| <code>MAX_EXPORT_SIGNING_KEYS</code>       | const             | <code>./private-state-provider</code>  |
| <code>MAX_EXPORT_STATES</code>             | const             | <code>./private-state-provider</code>  |
| <code>MidnightProvider</code>              | interface         | <code>./midnight-provider</code>       |
| <code>MidnightProviders</code>             | interface         | <code>./providers</code>               |
| <code>PrivateStateExport</code>            | interface         | <code>./private-state-provider</code>  |
| <code>PrivateStateExportError</code>       | class             | <code>./errors</code>                  |
| <code>PrivateStateId</code>                | type              | <code>./private-state-provider</code>  |
| <code>PrivateStateImportError</code>       | class             | <code>./errors</code>                  |
| <code>PrivateStateImportErrorCause</code>  | type              | <code>./errors</code>                  |
| <code>PrivateStateProvider</code>          | interface         | <code>./private-state-provider</code>  |
| <code>ProofProvider</code>                 | interface         | <code>./proof-provider</code>          |
| <code>ProverKey</code>                     | type              | <code>./midnight-types</code>          |
| <code>ProveTxConfig</code>                 | interface         | <code>./proof-provider</code>          |
| <code>PublicDataProvider</code>            | interface         | <code>./public-data-provider</code>    |
| <code>SegmentFail</code>                   | const             | <code>./midnight-types</code>          |
| <code>SegmentStatus</code>                 | type              | <code>./midnight-types</code>          |
| <code>SegmentSuccess</code>                | const             | <code>./midnight-types</code>          |
| <code>SigningKeyExport</code>              | interface         | <code>./private-state-provider</code>  |
| <code>SigningKeyExportError</code>         | class             | <code>./errors</code>                  |
| <code>SucceedEntirely</code>               | const             | <code>./midnight-types</code>          |
| <code>Transaction</code>                   | external-reexport | <code>@midnight-ntwrk/ledger-v8</code> |
| <code>TxIdConfig</code>                    | type              | <code>./public-data-provider</code>    |
| <code>TxStatus</code>                      | type              | <code>./midnight-types</code>          |
| <code>UnboundTransaction</code>            | type              | <code>./proof-provider</code>          |
| <code>UnshieldedBalance</code>             | type              | <code>./midnight-types</code>          |
| <code>UnshieldedBalances</code>            | type              | <code>./midnight-types</code>          |
| <code>UnshieldedUtxo</code>                | type              | <code>./midnight-types</code>          |
| <code>UnshieldedUtxos</code>               | type              | <code>./midnight-types</code>          |
| <code>VerifierKey</code>                   | type              | <code>./midnight-types</code>          |
| <code>WalletProvider</code>                | interface         | <code>./wallet-provider</code>         |
| <code>ZKConfig</code>                      | interface         | <code>./midnight-types</code>          |
| <code>ZKConfigProvider</code>              | abstract-class    | <code>./zk-config-provider</code>      |

| Export                                    | Kind  | Source                        |
|-------------------------------------------|-------|-------------------------------|
| <code>zkConfigToProvingKeyMaterial</code> | const | <code>./midnight-types</code> |
| <code>ZKIR</code>                         | type  | <code>./midnight-types</code> |

## `utils` (`@midnight-ntwrk/midnight-js-utils`)

| Export                               | Kind     | Source                         |
|--------------------------------------|----------|--------------------------------|
| <code>assertDefined</code>           | function | <code>./assertion-utils</code> |
| <code>assertIsContractAddress</code> | function | <code>./type-utils</code>      |
| <code>assertIsHex</code>             | function | <code>./hex-utils</code>       |
| <code>assertUndefined</code>         | function | <code>./assertion-utils</code> |
| <code>fromHex</code>                 | const    | <code>./hex-utils</code>       |
| <code>isHex</code>                   | const    | <code>./hex-utils</code>       |
| <code>parseCoinPublicKeyToHex</code> | const    | <code>./hex-utils</code>       |
| <code>ParsedHexString</code>         | type     | <code>./hex-utils</code>       |
| <code>parseEncPublicKeyToHex</code>  | const    | <code>./hex-utils</code>       |
| <code>parseHex</code>                | const    | <code>./hex-utils</code>       |
| <code>toHex</code>                   | const    | <code>./hex-utils</code>       |
| <code>ttlOneHour</code>              | const    | <code>./date-utils</code>      |

### Example usage `[deterministic]`

- `assertDefined` — [Example 7: Utility Functions, Type Helpers, and Network Configuration](#)
- `assertIsContractAddress` — [Example 4: Low-Level Transaction Pipeline](#), [Example 7: Utility Functions, Type Helpers, and Network Configuration](#)
- `assertIsHex` — [Example 7: Utility Functions, Type Helpers, and Network Configuration](#)
- `assertUndefined` — [Example 7: Utility Functions, Type Helpers, and Network Configuration](#)
- `fromHex` — [Example 4: Low-Level Transaction Pipeline](#), [Example 7: Utility Functions, Type Helpers, and Network Configuration](#)
- `isHex` — [Example 4: Low-Level Transaction Pipeline](#), [Example 7: Utility Functions, Type Helpers, and Network Configuration](#)
- `parseCoinPublicKeyToHex` — [Example 8: Advanced Deploy Pipeline and Provider Configuration](#), [Example 7: Utility Functions, Type Helpers, and Network Configuration](#)
- `parseEncPublicKeyToHex` — [Example 7: Utility Functions, Type Helpers, and Network Configuration](#)
- `parseHex` — [Example 7: Utility Functions, Type Helpers, and Network Configuration](#)
- `toHex` — [Example 4: Low-Level Transaction Pipeline](#), [Example 7: Utility Functions, Type Helpers, and Network Configuration](#)
- `ttlOneHour` — [Example 7: Utility Functions, Type Helpers, and Network Configuration](#)

## TypeScript Type Verification [deterministic]

14 type test modules compiled successfully with 389 type assertions (368 positive, 21 negative). Every assertion uses `tsc --noEmit` as the oracle — the TypeScript compiler is the source of truth for type correctness.

| Test Module                                | Types Tested                                                                     | Assertions   | Verdict  |
|--------------------------------------------|----------------------------------------------------------------------------------|--------------|----------|
| <code>branded-types.test.ts</code>         | ProverKey, VerifierKey, ZKIR (branded types)                                     | 29 (22+, 7–) | COMPILED |
| <code>contracts-call.test.ts</code>        | CallOptionsBase, CallOptionsWithArguments, CallOptionsProviderDataDependencies   | 40 (40+, 0–) | COMPILED |
| <code>contracts-constructor.test.ts</code> | ContractConstructorOptionsBase, ContractConstructorOptionsWithArguments          | 18 (18+, 0–) | COMPILED |
| <code>contracts-deploy.test.ts</code>      | DeployContractOptionsBase, DeployContractOptionsWithPrivateState                 | 41 (41+, 0–) | COMPILED |
| <code>contracts-errors.test.ts</code>      | TxFailedError, DeployTxFailedError, CallTxFailedError                            | 21 (19+, 2–) | COMPILED |
| <code>contracts-interfaces.test.ts</code>  | ContractProviders, PublicContractStates, ContractStates                          | 23 (23+, 0–) | COMPILED |
| <code>contracts-tx-data.test.ts</code>     | UnsubmittedTxData, UnsubmittedDeployTxPublicData, UnsubmittedDeployTxPrivateData | 36 (36+, 0–) | COMPILED |
| <code>error-classes.test.ts</code>         | InvalidProtocolSchemeError, PrivateStateExportError                              | 26 (24+, 2–) | COMPILED |
| <code>logging.test.ts</code>               | LogLevel (enum), LoggerProvider (interface).                                     | 16 (16+, 0–) | COMPILED |
| <code>observable-config.test.ts</code>     | All, Latest, TxIdConfig, BlockHeightConfig                                       | 19 (16+, 3–) | COMPILED |
| <code>private-state.test.ts</code>         | PrivateStateId, PrivateStateProvider, PrivateStateExport                         | 33 (31+, 2–) | COMPILED |
| <code>provider-interfaces.test.ts</code>   | MidnightProvider, ProofProvider, ProveTxConfig                                   | 37 (37+, 0–) | COMPILED |
| <code>transaction-status.test.ts</code>    | TxStatus, SegmentStatus, Fees, BlockHash, PrivateStateId                         | 35 (32+, 3–) | COMPILED |
| <code>zk-config-provider.test.ts</code>    | ZKConfigProvider (abstract class), KeyMaterialProvider                           | 15 (13+, 2–) | COMPILED |



## 3. Provider Interfaces

The seven provider interfaces define the contract between the SDK core and the pluggable implementations. Each interface is documented with its methods, the concrete implementation(s) available, and verification evidence.

### 3.1 PrivateStateProvider

Package: `@midnight-ntwrk/midnight-js-types` (interface) Implementation: `@midnight-ntwrk/midnight-js-level-private-state-provider`

#### Exports [deterministic]

| Export                                       | Kind      | Package                      |
|----------------------------------------------|-----------|------------------------------|
| <code>PrivateStateProvider</code>            | interface | types                        |
| <code>PrivateStateId</code>                  | type      | types                        |
| <code>ExportPrivateStatesOptions</code>      | interface | types                        |
| <code>ImportPrivateStatesOptions</code>      | interface | types                        |
| <code>ImportPrivateStatesResult</code>       | interface | types                        |
| <code>ExportSigningKeysOptions</code>        | interface | types                        |
| <code>ImportSigningKeysOptions</code>        | interface | types                        |
| <code>ImportSigningKeysResult</code>         | interface | types                        |
| <code>PrivateStateExport</code>              | interface | types                        |
| <code>SigningKeyExport</code>                | interface | types                        |
| <code>MAX_EXPORT_STATES</code>               | const     | types                        |
| <code>MAX_EXPORT_SIGNING_KEYS</code>         | const     | types                        |
| <code>levelPrivateStateProvider</code>       | const     | level-private-state-provider |
| <code>LevelPrivateStateProviderConfig</code> | type      | level-private-state-provider |
| <code>DEFAULT_CONFIG</code>                  | const     | level-private-state-provider |
| <code>StorageEncryption</code>               | class     | level-private-state-provider |
| <code>PrivateStoragePasswordProvider</code>  | type      | level-private-state-provider |
| <code>migrateToAccountScoped</code>          | const     | level-private-state-provider |
| <code>MigrationResult</code>                 | type      | level-private-state-provider |
| <code>PasswordRotationOptions</code>         | type      | level-private-state-provider |
| <code>PasswordRotationResult</code>          | type      | level-private-state-provider |
| <code>decryptValue</code>                    | const     | level-private-state-provider |

#### Description [AI-generated]

The `PrivateStateProvider` interface manages encrypted local state for each user. Private state is stored per-contract (keyed by `PrivateStateId`), encrypted at rest with AES-256-GCM, and scoped to wallet accounts.

The LevelDB implementation (`levelPrivateStateProvider`) uses PBKDF2 key derivation (600,000 iterations for V2) and provides CRUD operations (`get`, `set`, `remove`), bulk export/import with conflict resolution, and migration utilities for account-scoped isolation.

**Verification evidence** `[deterministic]`

| Test                                                  | Description   | Expected | Verdict |
|-------------------------------------------------------|---------------|----------|---------|
| factory creates provider                              | private-state | pass     | PASS    |
| provider has expected methods                         | private-state | pass     | PASS    |
| setContractAddress accepts valid address              | private-state | pass     | PASS    |
| get returns null for non-existent key                 | private-state | pass     | PASS    |
| set stores a string value                             | private-state | pass     | PASS    |
| get retrieves stored string                           | private-state | pass     | PASS    |
| set stores a number value                             | private-state | pass     | PASS    |
| get retrieves stored number                           | private-state | pass     | PASS    |
| set stores an object with BigInt                      | private-state | pass     | PASS    |
| get retrieves object with BigInt preserved            | private-state | pass     | PASS    |
| set stores Uint8Array                                 | private-state | pass     | PASS    |
| get retrieves Uint8Array                              | private-state | pass     | PASS    |
| remove deletes a key                                  | private-state | pass     | PASS    |
| other keys still exist after remove                   | private-state | pass     | PASS    |
| clear removes all keys                                | private-state | pass     | PASS    |
| getSigningKey returns null for unknown contract       | private-state | pass     | PASS    |
| setSigningKey stores a key                            | private-state | pass     | PASS    |
| getSigningKey retrieves stored key                    | private-state | pass     | PASS    |
| removeSigningKey deletes key                          | private-state | pass     | PASS    |
| exportPrivateStates produces encrypted export         | private-state | pass     | PASS    |
| importPrivateStates restores from export              | private-state | pass     | PASS    |
| imported data is accessible                           | private-state | pass     | PASS    |
| short password rejected                               | private-state | pass     | PASS    |
| migrateToAccountScoped is a function                  | private-state | pass     | PASS    |
| migrateToAccountScoped requires accountId             | private-state | pass     | PASS    |
| migrateToAccountScoped rejects empty accountId        | private-state | pass     | PASS    |
| migrateToAccountScoped returns migration counts       | private-state | pass     | PASS    |
| different accountIds produce isolated storage         | private-state | pass     | PASS    |
| same accountId always maps to same scoped location    | private-state | pass     | PASS    |
| migrateToAccountScoped is idempotent (safe to re-run) | private-state | pass     | PASS    |
| DEFAULT_CONFIG has midnightDbName field               | private-state | pass     | PASS    |
| DEFAULT_CONFIG has privateStateStoreName field        | private-state | pass     | PASS    |
| DEFAULT_CONFIG has signingKeyStoreName field          | private-state | pass     | PASS    |

| Test                                                                             | Description   | Expected | Verdict |
|----------------------------------------------------------------------------------|---------------|----------|---------|
| StorageEncryption is constructable with password                                 | private-state | pass     | PASS    |
| decryptValue is a function                                                       | private-state | pass     | PASS    |
| migrateToAccountScoped is a function                                             | private-state | pass     | PASS    |
| StorageEncryption produces base64 output                                         | private-state | pass     | PASS    |
| encrypted output has correct header structure (version + salt + IV + authTag)    | private-state | pass     | PASS    |
| version byte is 2 (current encryption version)                                   | private-state | pass     | PASS    |
| IV is 12 bytes (AES-GCM standard nonce length)                                   | private-state | pass     | PASS    |
| auth tag is 16 bytes (AES-GCM standard tag length)                               | private-state | pass     | PASS    |
| different encryptions of same plaintext produce different ciphertext (random IV) | private-state | pass     | PASS    |
| encrypt/decrypt roundtrip preserves plaintext                                    | private-state | pass     | PASS    |
| decrypt with wrong password fails (auth tag verification)                        | private-state | pass     | PASS    |
| StorageEncryption.isEncrypted detects encrypted data                             | private-state | pass     | PASS    |
| StorageEncryption.isEncrypted rejects plain text                                 | private-state | pass     | PASS    |
| StorageEncryption.getVersion returns 2 for current encryption                    | private-state | pass     | PASS    |
| StorageEncryption constructor takes measurable time (>50ms for 600K PBKDF2)      | private-state | pass     | PASS    |
| encryption version prefix confirms v2 (600K iterations path)                     | private-state | pass     | PASS    |
| salt is 32 bytes and unique per StorageEncryption instance                       | private-state | pass     | PASS    |
| verifyPassword confirms correct password via timing-safe comparison              | private-state | pass     | PASS    |
| verifyPassword rejects wrong password                                            | private-state | pass     | PASS    |

#### Example usage [deterministic]

- `PrivateStateProvider` — [Example 1: Counter Contract — Deploy, Call, Observe](#)
- `MAX_EXPORT_STATES` — [Example 7: Utility Functions, Type Helpers, and Network Configuration](#)
- `MAX_EXPORT_SIGNING_KEYS` — [Example 7: Utility Functions, Type Helpers, and Network Configuration](#)
- `levelPrivateStateProvider` — [Example 8: Advanced Deploy Pipeline and Provider Configuration](#), [Example 5: Contract Maintenance — Authority and Verifier Keys](#), [Example 1: Counter Contract — Deploy, Call, Observe](#), [Example 4: Low-Level Transaction Pipeline](#), [Example 2: Private Counter — Witnesses and Authorization](#), [Example 6: Private State — Encryption, Backup, and Migration](#), [Example 3: State Observation — Queries, Verification, and Observables](#)
- `DEFAULT_CONFIG` — [Example 6: Private State — Encryption, Backup, and Migration](#)
- `StorageEncryption` — [Example 6: Private State — Encryption, Backup, and Migration](#)
- `migrateToAccountScoped` — [Example 6: Private State — Encryption, Backup, and Migration](#)
- `decryptValue` — [Example 6: Private State — Encryption, Backup, and Migration](#)

## 3.2 PublicDataProvider

Package: `@midnight-ntwrk/midnight-js-types` (interface) Implementation: `@midnight-ntwrk/midnight-js-indexer-public-data-provider`

### Exports [deterministic]

| Export                                     | Kind      | Package                      |
|--------------------------------------------|-----------|------------------------------|
| <code>PublicDataProvider</code>            | interface | types                        |
| <code>All</code>                           | type      | types                        |
| <code>Latest</code>                        | type      | types                        |
| <code>BlockHashConfig</code>               | type      | types                        |
| <code>BlockHeightConfig</code>             | type      | types                        |
| <code>TxIdConfig</code>                    | type      | types                        |
| <code>ContractStateObservableConfig</code> | type      | types                        |
| <code>indexerPublicDataProvider</code>     | const     | indexer-public-data-provider |
| <code>IndexerUtxo</code>                   | type      | indexer-public-data-provider |
| <code>IndexerFormattedError</code>         | class     | indexer-public-data-provider |
| <code>toUnshieldedBalances</code>          | const     | indexer-public-data-provider |
| <code>toUnshieldedUtxos</code>             | const     | indexer-public-data-provider |

### Description [AI-generated]

The `PublicDataProvider` interface queries on-chain state: contract state, transaction status, block data, and unshielded balances. It supports polling ( `watchForContractState` ) and real-time subscriptions ( `contractStateObservable` ).

The indexer implementation uses GraphQL (Apollo Client) over HTTP for queries and WebSocket for subscriptions. Query scope is configurable via `All` , `Latest` , `BlockHashConfig` , `BlockHeightConfig` , or `TxIdConfig` .

### Verification evidence [deterministic]

| Test                                                       | Description | Expected | Verdict |
|------------------------------------------------------------|-------------|----------|---------|
| toUnshieldedBalances transforms empty array                | indexer     | pass     | PASS    |
| toUnshieldedUtxos transforms empty arrays                  | indexer     | pass     | PASS    |
| indexerPublicDataProvider factory creates provider object  | indexer     | pass     | PASS    |
| indexerPublicDataProvider has query and observable methods | indexer     | pass     | PASS    |

### Example usage [deterministic]

- `PublicDataProvider` — [Example 1: Counter Contract — Deploy, Call, Observe](#)
- `All` — [Example 8: Advanced Deploy Pipeline and Provider Configuration](#), [Example 5: Contract Maintenance — Authority and Verifier Keys](#), [Example 1: Counter Contract — Deploy, Call, Observe](#), [Example 4: Low-Level Transaction Pipeline](#), [Example 2: Private Counter — Witnesses and Authorization](#), [Example 3: State Observation — Queries, Verification, and Observables](#)

- `indexerPublicDataProvider` — [Example 8: Advanced Deploy Pipeline and Provider Configuration](#), [Example 5: Contract Maintenance — Authority and Verifier Keys](#), [Example 1: Counter Contract — Deploy, Call, Observe](#), [Example 4: Low-Level Transaction Pipeline](#), [Example 2: Private Counter — Witnesses and Authorization](#), [Example 3: State Observation — Queries, Verification, and Observables](#)
- `IndexerFormattedError` — [Example 8: Advanced Deploy Pipeline and Provider Configuration](#)
- `toUnshieldedBalances` — [Example 8: Advanced Deploy Pipeline and Provider Configuration](#)
- `toUnshieldedUtxos` — [Example 8: Advanced Deploy Pipeline and Provider Configuration](#)

### 3.3 ZKConfigProvider

Package: `@midnight-ntwrk/midnight-js-types` (abstract class) Implementations: `fetch-zk-config-provider`, `node-zk-config-provider`

Exports [deterministic]

| Export                                    | Kind           | Package                  |
|-------------------------------------------|----------------|--------------------------|
| <code>ZKConfigProvider</code>             | abstract-class | types                    |
| <code>KeyMaterialProvider</code>          | type           | types                    |
| <code>ZKConfig</code>                     | interface      | types                    |
| <code>ZKIR</code>                         | type           | types                    |
| <code>ProverKey</code>                    | type           | types                    |
| <code>VerifierKey</code>                  | type           | types                    |
| <code>createProverKey</code>              | const          | types                    |
| <code>createVerifierKey</code>            | const          | types                    |
| <code>createZKIR</code>                   | const          | types                    |
| <code>zkConfigToProvingKeyMaterial</code> | const          | types                    |
| <code>FetchZkConfigProvider</code>        | class          | fetch-zk-config-provider |
| <code>NodeZkConfigProvider</code>         | class          | node-zk-config-provider  |

Description [AI-generated]

The `ZKConfigProvider` abstract class retrieves zero-knowledge artifacts: ZKIR circuit descriptions, prover keys, and verifier keys. These are the cryptographic materials needed for proof generation and on-chain verification.

`FetchZkConfigProvider` uses browser `fetch()` to download artifacts from a URL. `NodeZkConfigProvider` reads them from the local filesystem. Neither provider caches artifacts — every call to `getProverKey`, `getVerifierKey`, or `getZKIR` hits the backing store (network or filesystem) directly.

Verification evidence [deterministic]

| Test                                                                          | Description | Expected | Verdict |
|-------------------------------------------------------------------------------|-------------|----------|---------|
| createProverKey returns input Uint8Array                                      | types       | pass     | PASS    |
| createVerifierKey returns input Uint8Array                                    | types       | pass     | PASS    |
| createZKIR returns input Uint8Array                                           | types       | pass     | PASS    |
| zkConfigToProvingKeyMaterial maps fields correctly                            | types       | pass     | PASS    |
| ZKConfigProvider is an abstract class with methods                            | types       | pass     | PASS    |
| ZKConfigProvider is instantiable (base class)                                 | types       | pass     | PASS    |
| FetchZkConfigProvider instantiates with URL string                            | zk-config   | pass     | PASS    |
| FetchZkConfigProvider has getProverKey/getVerifierKey/getZKIR methods         | zk-config   | pass     | PASS    |
| NodeZkConfigProvider instantiates with path string                            | zk-config   | pass     | PASS    |
| NodeZkConfigProvider has readFile/getProverKey/getVerifierKey/getZKIR methods | zk-config   | pass     | PASS    |
| NodeZkConfigProvider constructor accepts directory path                       | zk-config   | pass     | PASS    |
| provider has getProverKey, getVerifierKey, getZKIR methods                    | zk-config   | pass     | PASS    |
| provider has inherited get() and getVerifierKeys() from ZKConfigProvider      | zk-config   | pass     | PASS    |
| getProverKey reads keys/.prover                                               | zk-config   | pass     | PASS    |
| getVerifierKey reads keys/.verifier                                           | zk-config   | pass     | PASS    |
| getZKIR reads zkir/.bzkir                                                     | zk-config   | pass     | PASS    |
| getProverKey throws for nonexistent circuit                                   | zk-config   | pass     | PASS    |
| getVerifierKey throws for nonexistent circuit                                 | zk-config   | pass     | PASS    |
| getZKIR throws for nonexistent circuit                                        | zk-config   | pass     | PASS    |
| get() returns all three artifacts for a circuit                               | zk-config   | pass     | PASS    |
| getVerifierKeys returns array of [id, key] pairs                              | zk-config   | pass     | PASS    |
| FetchZkConfigProvider validates URL protocol (rejects ftp://)                 | zk-config   | pass     | PASS    |
| FetchZkConfigProvider accepts http:// URL                                     | zk-config   | pass     | PASS    |
| FetchZkConfigProvider accepts https:// URL                                    | zk-config   | pass     | PASS    |
| FetchZkConfigProvider fetches prover key from keys/.prover                    | zk-config   | pass     | PASS    |
| FetchZkConfigProvider fetches verifier key from keys/.verifier                | zk-config   | pass     | PASS    |
| FetchZkConfigProvider fetches ZKIR from zkir/.bzkir                           | zk-config   | pass     | PASS    |
| FetchZkConfigProvider throws on HTTP error response                           | zk-config   | pass     | PASS    |
| FetchZkConfigProvider hits network on every call (no cache)                   | zk-config   | pass     | PASS    |
| NodeZkConfigProvider has no cache-related properties                          | zk-config   | pass     | PASS    |

#### Example usage [deterministic]

- `ZKConfigProvider` — [Example 8: Advanced Deploy Pipeline and Provider Configuration](#)
- `createProverKey` — [Example 7: Utility Functions, Type Helpers, and Network Configuration](#)

- `createVerifierKey` — [Example 7: Utility Functions, Type Helpers, and Network Configuration](#)
- `createZKIR` — [Example 7: Utility Functions, Type Helpers, and Network Configuration](#)
- `zkConfigToProvingKeyMaterial` — [Example 8: Advanced Deploy Pipeline and Provider Configuration](#)
- `FetchZkConfigProvider` — [Example 8: Advanced Deploy Pipeline and Provider Configuration](#)
- `NodeZkConfigProvider` — [Example 8: Advanced Deploy Pipeline and Provider Configuration](#), [Example 5: Contract Maintenance — Authority and Verifier Keys](#), [Example 1: Counter Contract — Deploy, Call, Observe](#), [Example 4: Low-Level Transaction Pipeline](#), [Example 2: Private Counter — Witnesses and Authorization](#), [Example 3: State Observation — Queries, Verification, and Observables](#)

## 3.4 ProofProvider

Package: `@midnight-ntwrk/midnight-js-types` (interface) Implementation: `@midnight-ntwrk/midnight-js-http-client-proof-provider`

### Exports [deterministic]

| Export                                 | Kind      | Package                    |
|----------------------------------------|-----------|----------------------------|
| <code>ProofProvider</code>             | interface | types                      |
| <code>ProveTxConfig</code>             | interface | types                      |
| <code>UnboundTransaction</code>        | type      | types                      |
| <code>httpClientProofProvider</code>   | const     | http-client-proof-provider |
| <code>DEFAULT_CONFIG</code>            | const     | http-client-proof-provider |
| <code>httpClientProvingProvider</code> | const     | http-client-proof-provider |
| <code>DEFAULT_TIMEOUT</code>           | const     | http-client-proof-provider |
| <code>ProvingProviderConfig</code>     | type      | http-client-proof-provider |

### Description [AI-generated]

The `ProofProvider` interface takes an unproven transaction and produces a proven transaction by generating zero-knowledge proofs for each circuit call. The proof server is a black-box oracle: unproven transaction in, proven transaction out (or error).

The HTTP client implementation sends the unproven transaction to a proof server endpoint (default: port 6300) and returns the proven result. It also provides a lower-level `ProvingProvider` for direct proof generation.

### Verification evidence [deterministic]



| Test                                                                       | Description    | Expected | Verdict |
|----------------------------------------------------------------------------|----------------|----------|---------|
| DEFAULT_TIMEOUT is 300000ms (5 minutes)                                    | proof-provider | pass     | PASS    |
| DEFAULT_CONFIG contains timeout field                                      | proof-provider | pass     | PASS    |
| DEFAULT_TIMEOUT is exported                                                | proof-provider | pass     | PASS    |
| DEFAULT_TIMEOUT equals 300000 (5 minutes in ms)                            | proof-provider | pass     | PASS    |
| DEFAULT_TIMEOUT is exactly 5 minutes                                       | proof-provider | pass     | PASS    |
| DEFAULT_CONFIG is exported                                                 | proof-provider | pass     | PASS    |
| DEFAULT_CONFIG.timeout equals 300000                                       | proof-provider | pass     | PASS    |
| DEFAULT_CONFIG.timeout matches DEFAULT_TIMEOUT                             | proof-provider | pass     | PASS    |
| source defines retryOptions with retries: 3                                | proof-provider | pass     | PASS    |
| source defines retryOn: [500, 503]                                         | proof-provider | pass     | PASS    |
| source defines exponential backoff: $2^{\text{attempt}} * 1000\text{ms}$   | proof-provider | pass     | PASS    |
| exponential backoff formula: attempt 0 = 1000ms                            | proof-provider | pass     | PASS    |
| exponential backoff formula: attempt 1 = 2000ms                            | proof-provider | pass     | PASS    |
| exponential backoff formula: attempt 2 = 4000ms                            | proof-provider | pass     | PASS    |
| total maximum retry delay: $1\text{s} + 2\text{s} + 4\text{s} = 7\text{s}$ | proof-provider | pass     | PASS    |
| source imports fetchBuilder from fetch-retry                               | proof-provider | pass     | PASS    |
| source wraps fetch with fetchBuilder(fetch, retryOptions)                  | proof-provider | pass     | PASS    |
| fetch-retry package is installed                                           | proof-provider | pass     | PASS    |
| source uses AbortSignal.timeout(timeout) for request timeout               | proof-provider | pass     | PASS    |
| timeout defaults to DEFAULT_TIMEOUT when config not provided               | proof-provider | pass     | PASS    |
| httpClientProofProvider is a function                                      | proof-provider | pass     | PASS    |
| httpClientProvingProvider is a function                                    | proof-provider | pass     | PASS    |
| httpClientProvingProvider rejects ftp:// protocol                          | proof-provider | pass     | PASS    |
| httpClientProvingProvider accepts http:// protocol                         | proof-provider | pass     | PASS    |
| httpClientProvingProvider returns object with check and prove methods      | proof-provider | pass     | PASS    |
| httpClientProofProvider returns object with proveTx method                 | proof-provider | pass     | PASS    |

#### Example usage [deterministic]

- `ProofProvider` — [Example 1: Counter Contract — Deploy, Call, Observe](#)
- `httpClientProofProvider` — [Example 8: Advanced Deploy Pipeline and Provider Configuration](#), [Example 5: Contract Maintenance — Authority and Verifier Keys](#), [Example 1: Counter Contract — Deploy, Call, Observe](#), [Example 4: Low-Level Transaction Pipeline](#), [Example 2: Private Counter — Witnesses and Authorization](#), [Example 3: State Observation — Queries, Verification, and Observables](#)
- `DEFAULT_CONFIG` — [Example 6: Private State — Encryption, Backup, and Migration](#)
- `httpClientProvingProvider` — [Example 8: Advanced Deploy Pipeline and Provider Configuration](#)
- `DEFAULT_TIMEOUT` — [Example 8: Advanced Deploy Pipeline and Provider Configuration](#)

### 3.5 WalletProvider

Package: @midnight-ntwrk/midnight-js-types (interface)

Exports [deterministic]

| Export         | Kind      | Package |
|----------------|-----------|---------|
| WalletProvider | interface | types   |

Description [AI-generated]

The `WalletProvider` interface balances and signs transactions. After proof generation, the wallet adds the necessary inputs (shielded and unshielded tokens) to cover fees and any token transfers, then signs the complete transaction.

No implementation is provided by the midnight-js SDK itself -- the wallet provider comes from `@midnight-ntwrk/wallet-sdk-facade` (a separate package). No verification evidence is possible for this interface within the SDK scope.

Example usage [deterministic]

- `WalletProvider` — [Example 1: Counter Contract — Deploy, Call, Observe](#)

### 3.6 MidnightProvider

Package: @midnight-ntwrk/midnight-js-types (interface)

Exports [deterministic]

| Export           | Kind      | Package |
|------------------|-----------|---------|
| MidnightProvider | interface | types   |

Description [AI-generated]

The `MidnightProvider` interface submits balanced, proven, signed transactions to the Midnight network. It is the final step in the transaction pipeline.

Like `WalletProvider`, the implementation comes from the wallet facade package, not from the midnight-js SDK itself. No verification evidence is possible for this interface within the SDK scope.

Example usage [deterministic]

- `MidnightProvider` — [Example 1: Counter Contract — Deploy, Call, Observe](#)

### 3.7 LoggerProvider

Package: @midnight-ntwrk/midnight-js-types (interface) Implementation: @midnight-ntwrk/midnight-js-logger-provider

Exports [deterministic]

| Export            | Kind      | Package         |
|-------------------|-----------|-----------------|
| MidnightProviders | interface | types           |
| LoggerProvider    | interface | types           |
| LogLevel          | enum      | types           |
| LoggerProvider    | class     | logger-provider |

Description [AI-generated]

Optional structured logging interface. The implementation wraps Pino. `LogLevel` enum provides: `trace`, `debug`, `info`, `warn`, `error`, `fatal`, `silent`.

Verification evidence [deterministic]

| Test                            | Description | Expected | Verdict |
|---------------------------------|-------------|----------|---------|
| LogLevel has expected values    | types       | pass     | PASS    |
| LoggerProvider is constructable | logger      | pass     | PASS    |

Example usage [deterministic]

- `LoggerProvider` — [Example 8: Advanced Deploy Pipeline and Provider Configuration](#), [Example 7: Utility Functions, Type Helpers, and Network Configuration](#)
- `LogLevel` — [Example 7: Utility Functions, Type Helpers, and Network Configuration](#)
- `LoggerProvider` — [Example 8: Advanced Deploy Pipeline and Provider Configuration](#), [Example 7: Utility Functions, Type Helpers, and Network Configuration](#)

## 4. Contract Lifecycle

The core SDK workflow: deploy a contract, interact with it, and observe state changes.

### 4.1 Deploying a Contract

**Key exports** [deterministic]

| Export                                                              | Kind     | Source                       |
|---------------------------------------------------------------------|----------|------------------------------|
| <code>deployContract</code>                                         | function | contracts/deploy-contract    |
| <code>DeployContractOptions</code>                                  | type     | contracts/deploy-contract    |
| <code>DeployContractOptionsBase</code>                              | type     | contracts/deploy-contract    |
| <code>DeployContractOptionsWithPrivateState</code>                  | type     | contracts/deploy-contract    |
| <code>DeployedContract</code>                                       | type     | contracts/deploy-contract    |
| <code>ContractConstructorOptions</code>                             | type     | contracts/call-constructor   |
| <code>ContractConstructorOptionsBase</code>                         | type     | contracts/call-constructor   |
| <code>ContractConstructorOptionsProviderDataDependencies</code>     | type     | contracts/call-constructor   |
| <code>ContractConstructorOptionsWithArguments</code>                | type     | contracts/call-constructor   |
| <code>ContractConstructorOptionsWithPrivateState</code>             | type     | contracts/call-constructor   |
| <code>ContractConstructorOptionsWithProviderDataDependencies</code> | type     | contracts/call-constructor   |
| <code>ContractConstructorResult</code>                              | type     | contracts/call-constructor   |
| <code>createUnprovenDeployTx</code>                                 | function | contracts/unproven-deploy-tx |
| <code>createUnprovenDeployTxFromVerifierKeys</code>                 | function | contracts/unproven-deploy-tx |
| <code>UnprovenDeployTxOptions</code>                                | type     | contracts/unproven-deploy-tx |
| <code>UnprovenDeployTxProviders</code>                              | type     | contracts/unproven-deploy-tx |
| <code>DeployTxOptionsBase</code>                                    | type     | contracts/unproven-deploy-tx |
| <code>DeployTxOptionsWithPrivateState</code>                        | type     | contracts/unproven-deploy-tx |
| <code>DeployTxOptionsWithPrivateStateId</code>                      | type     | contracts/unproven-deploy-tx |
| <code>submitDeployTx</code>                                         | function | contracts/submit-deploy-tx   |
| <code>DeployTxOptions</code>                                        | type     | contracts/submit-deploy-tx   |

**Description** [AI-generated]

`deployContract` is the high-level entry point: it creates an unproven deploy transaction, proves it, balances it, submits it, and waits for finalization. The lower-level functions ( `createUnprovenDeployTx` , `submitDeployTx` ) expose individual pipeline stages.

The deploy transaction includes the contract's compiled code, verifier keys, initial ledger state, and initial private state.

**Verification evidence** [deterministic]

| Test                                                 | Description | Expected | Verdict |
|------------------------------------------------------|-------------|----------|---------|
| DeployTxFailedError extends Error                    | contracts   | pass     | PASS    |
| DeployTxFailedError extends TxFailedError            | contracts   | pass     | PASS    |
| createUnprovenDeployTx is a function                 | contracts   | pass     | PASS    |
| createUnprovenDeployTxFromVerifierKeys is a function | contracts   | pass     | PASS    |
| submitDeployTx is a function                         | contracts   | pass     | PASS    |
| findDeployedContract is a function                   | contracts   | pass     | PASS    |

#### Example usage [deterministic]

- `deployContract` — [Example 8: Advanced Deploy Pipeline and Provider Configuration](#), [Example 5: Contract Maintenance — Authority and Verifier Keys](#), [Example 1: Counter Contract — Deploy, Call, Observe](#), [Example 4: Low-Level Transaction Pipeline](#), [Example 2: Private Counter — Witnesses and Authorization](#), [Example 3: State Observation — Queries, Verification, and Observables](#)
- `createUnprovenDeployTx` — [Example 8: Advanced Deploy Pipeline and Provider Configuration](#)
- `createUnprovenDeployTxFromVerifierKeys` — [Example 8: Advanced Deploy Pipeline and Provider Configuration](#)
- `submitDeployTx` — [Example 8: Advanced Deploy Pipeline and Provider Configuration](#)

## 4.2 Finding a Deployed Contract

#### Key exports [deterministic]

| Export                                                       | Kind     | Source                           |
|--------------------------------------------------------------|----------|----------------------------------|
| <code>findDeployedContract</code>                            | function | contracts/find-deployed-contract |
| <code>FindDeployedContractOptions</code>                     | type     | contracts/find-deployed-contract |
| <code>FindDeployedContractOptionsBase</code>                 | type     | contracts/find-deployed-contract |
| <code>FindDeployedContractOptionsExistingPrivateState</code> | type     | contracts/find-deployed-contract |
| <code>FindDeployedContractOptionsStorePrivateState</code>    | type     | contracts/find-deployed-contract |
| <code>FoundContract</code>                                   | type     | contracts/find-deployed-contract |
| <code>verifierKeysEqual</code>                               | const    | contracts/find-deployed-contract |
| <code>verifyContractState</code>                             | const    | contracts/find-deployed-contract |

#### Description [AI-generated]

`findDeployedContract` reconnects to an already-deployed contract. It loads the on-chain state, verifies the verifier keys match the expected compiled contract, and returns a `FoundContract` that can be called like a freshly deployed one.

#### Verification evidence [deterministic]

| Test                                                         | Description | Expected | Verdict |
|--------------------------------------------------------------|-------------|----------|---------|
| verifierKeysEqual: identical arrays return true              | contracts   | pass     | PASS    |
| verifierKeysEqual: different arrays return false             | contracts   | pass     | PASS    |
| verifierKeysEqual: different lengths return false            | contracts   | pass     | PASS    |
| verifierKeysEqual: empty arrays are equal                    | contracts   | pass     | PASS    |
| verifyContractState: matching keys does not throw            | contracts   | pass     | PASS    |
| verifyContractState: mismatched key throws ContractTypeError | contracts   | pass     | PASS    |
| verifyContractState: missing circuit on-chain throws         | contracts   | pass     | PASS    |

**Example usage** [deterministic]

- `findDeployedContract` — [Example 5: Contract Maintenance — Authority and Verifier Keys](#), [Example 1: Counter Contract — Deploy, Call, Observe](#), [Example 4: Low-Level Transaction Pipeline](#), [Example 2: Private Counter — Witnesses and Authorization](#), [Example 3: State Observation — Queries, Verification, and Observables](#)
- `verifierKeysEqual` — [Example 3: State Observation — Queries, Verification, and Observables](#)
- `verifyContractState` — [Example 3: State Observation — Queries, Verification, and Observables](#)

## 4.3 Calling a Circuit

**Key exports** [deterministic]

| Export                                               | Kind     | Source                     |
|------------------------------------------------------|----------|----------------------------|
| <code>CallOptions</code>                             | type     | contracts/call             |
| <code>CallOptionsBase</code>                         | type     | contracts/call             |
| <code>CallOptionsProviderDataDependencies</code>     | type     | contracts/call             |
| <code>CallOptionsWithArguments</code>                | type     | contracts/call             |
| <code>CallOptionsWithPrivateState</code>             | type     | contracts/call             |
| <code>CallOptionsWithProviderDataDependencies</code> | type     | contracts/call             |
| <code>CallResult</code>                              | type     | contracts/call             |
| <code>CallResultPublic</code>                        | type     | contracts/call             |
| <code>CallResultPrivate</code>                       | type     | contracts/call             |
| <code>createUnprovenCallTx</code>                    | function | contracts/unproven-call-tx |
| <code>createUnprovenCallTxFromInitialStates</code>   | function | contracts/unproven-call-tx |
| <code>CallTxOptions</code>                           | type     | contracts/unproven-call-tx |
| <code>CallTxOptionsBase</code>                       | type     | contracts/unproven-call-tx |
| <code>CallTxOptionsWithPrivateStateId</code>         | type     | contracts/unproven-call-tx |
| <code>UnprovenCallTxProvidersBase</code>             | type     | contracts/unproven-call-tx |
| <code>UnprovenCallTxProvidersWithPrivateState</code> | type     | contracts/unproven-call-tx |
| <code>submitCallTx</code>                            | function | contracts/submit-call-tx   |
| <code>submitCallTxAsync</code>                       | function | contracts/submit-call-tx   |

**Description** [AI-generated]

Circuit calls are the primary interaction mechanism. The high-level `callTx` interface (returned by `deployContract` / `findDeployedContract`) wraps the full pipeline: create unproven → prove → balance → submit → wait. `createUnprovenCallTxFromInitialStates` is the key function for local testing: it executes a circuit locally against provided initial states, producing proof data without any network interaction. This is what our execution oracle uses.

**Verification evidence** [deterministic]

| Test                                                    | Description | Expected | Verdict |
|---------------------------------------------------------|-------------|----------|---------|
| IncompleteCallTxPrivateStateConfig extends Error        | contracts   | pass     | PASS    |
| IncompleteCallTxPrivateStateConfig has specific message | contracts   | pass     | PASS    |
| createCallTxOptions creates options with all fields     | contracts   | pass     | PASS    |
| createCircuitCallTxInterface is a function              | contracts   | pass     | PASS    |
| submitCallTxAsync is a function                         | contracts   | pass     | PASS    |

**Example usage** [deterministic]

- `createUnprovenCallTx` — [Example 8: Advanced Deploy Pipeline and Provider Configuration](#), [Example 4: Low-Level Transaction Pipeline](#)
- `createUnprovenCallTxFromInitialStates` — [Example 8: Advanced Deploy Pipeline and Provider Configuration](#)

- `submitCallTx` — [Example 8: Advanced Deploy Pipeline and Provider Configuration](#), [Example 4: Low-Level Transaction Pipeline](#)
- `submitCallTxAsync` — [Example 4: Low-Level Transaction Pipeline](#)

## 4.4 Querying State

Key exports `[deterministic]`

| Export                             | Kind  | Source                            |
|------------------------------------|-------|-----------------------------------|
| <code>getStates</code>             | const | contracts/get-states              |
| <code>getPublicStates</code>       | const | contracts/get-states              |
| <code>getUnshieldedBalances</code> | const | contracts/get-unshielded-balances |
| <code>ContractStates</code>        | type  | contracts/get-states              |
| <code>PublicContractStates</code>  | type  | contracts/get-states              |

Description `[AI-generated]`

State query functions retrieve the current contract state. `getStates` returns both public and private state; `getPublicStates` returns only on-chain state (no private state provider needed); `getUnshieldedBalances` returns token balances.

Verification evidence `[deterministic]`

| Test                                                                  | Description | Expected | Verdict |
|-----------------------------------------------------------------------|-------------|----------|---------|
| <code>getPublicStates</code> : returns structured state from provider | contracts   | pass     | PASS    |
| <code>getPublicStates</code> : validates contract address             | contracts   | pass     | PASS    |
| <code>getStates</code> : returns public + private state               | contracts   | pass     | PASS    |
| <code>getStates</code> : throws when private state is missing         | contracts   | pass     | PASS    |
| <code>getUnshieldedBalances</code> : returns provider result          | contracts   | pass     | PASS    |
| <code>getUnshieldedBalances</code> : validates contract address       | contracts   | pass     | PASS    |

Example usage `[deterministic]`

- `getStates` — [Example 3: State Observation — Queries, Verification, and Observables](#)
- `getPublicStates` — [Example 3: State Observation — Queries, Verification, and Observables](#)
- `getUnshieldedBalances` — [Example 3: State Observation — Queries, Verification, and Observables](#)

## 4.5 Transaction Submission

Key exports `[deterministic]`



| Export                                 | Kind  | Source                                |
|----------------------------------------|-------|---------------------------------------|
| <code>submitTx</code>                  | const | contracts/submit-tx                   |
| <code>submitTxAsync</code>             | const | contracts/submit-tx                   |
| <code>SubmitTxOptions</code>           | type  | contracts/submit-tx                   |
| <code>SubmitTxProviders</code>         | type  | contracts/submit-tx                   |
| <code>submitInsertVerifierKeyTx</code> | const | contracts/submit-insert-vk-tx         |
| <code>submitRemoveVerifierKeyTx</code> | const | contracts/submit-remove-vk-tx         |
| <code>submitReplaceAuthorityTx</code>  | const | contracts/submit-replace-authority-tx |

### Description [AI-generated]

Low-level transaction submission functions. `submitTx` is the generic pipeline: prove → balance → submit → wait.

`submitTxAsync` submits without waiting for finalization.

Maintenance transactions manage verifier keys and contract authorities -- operations that don't involve circuit execution but modify contract metadata on-chain.

### Verification evidence [deterministic]

| Test                                                               | Description | Expected | Verdict |
|--------------------------------------------------------------------|-------------|----------|---------|
| <code>submitTxAsync</code> is a function                           | contracts   | pass     | PASS    |
| <code>submitInsertVerifierKeyTx</code> is a function               | contracts   | pass     | PASS    |
| <code>submitRemoveVerifierKeyTx</code> is a function               | contracts   | pass     | PASS    |
| <code>submitReplaceAuthorityTx</code> is a function                | contracts   | pass     | PASS    |
| <code>submitReplaceAuthorityTx</code> returns a function (curried) | contracts   | pass     | PASS    |

### Example usage [deterministic]

- `submitTx` — [Example 8: Advanced Deploy Pipeline and Provider Configuration](#), [Example 5: Contract Maintenance — Authority and Verifier Keys](#), [Example 1: Counter Contract — Deploy, Call, Observe](#), [Example 4: Low-Level Transaction Pipeline](#), [Example 2: Private Counter — Witnesses and Authorization](#), [Example 3: State Observation — Queries, Verification, and Observables](#)
- `submitTxAsync` — [Example 8: Advanced Deploy Pipeline and Provider Configuration](#)
- `submitInsertVerifierKeyTx` — [Example 5: Contract Maintenance — Authority and Verifier Keys](#)
- `submitRemoveVerifierKeyTx` — [Example 5: Contract Maintenance — Authority and Verifier Keys](#)
- `submitReplaceAuthorityTx` — [Example 5: Contract Maintenance — Authority and Verifier Keys](#)

## 5. Transaction Model

The types that describe transactions at each pipeline stage.

### 5.1 Transaction Context and Scoping

**Key exports** [deterministic]

| Export                                     | Kind      | Source                |
|--------------------------------------------|-----------|-----------------------|
| <code>TransactionContext</code>            | interface | contracts/transaction |
| <code>ScopedTransactionOptions</code>      | type      | contracts/transaction |
| <code>withContractScopedTransaction</code> | const     | contracts/transaction |

**Description** [AI-generated]

`TransactionContext` carries the state needed during a transaction's lifecycle. `withContractScopedTransaction` provides an atomic scope for multi-step operations — batching multiple circuit calls into a single transaction.

**Example usage** [deterministic]

- `withContractScopedTransaction` — [Example 8: Advanced Deploy Pipeline and Provider Configuration](#)

### 5.2 Transaction Interfaces

**Key exports** [deterministic]

| Export                                            | Kind      | Source                  |
|---------------------------------------------------|-----------|-------------------------|
| <code>CircuitCallTxInterface</code>               | type      | contracts/tx-interfaces |
| <code>CircuitMaintenanceTxInterface</code>        | type      | contracts/tx-interfaces |
| <code>CircuitMaintenanceTxInterfaces</code>       | type      | contracts/tx-interfaces |
| <code>ContractMaintenanceTxInterface</code>       | interface | contracts/tx-interfaces |
| <code>createCallTxOptions</code>                  | const     | contracts/tx-interfaces |
| <code>createCircuitCallTxInterface</code>         | const     | contracts/tx-interfaces |
| <code>createCircuitMaintenanceTxInterface</code>  | const     | contracts/tx-interfaces |
| <code>createCircuitMaintenanceTxInterfaces</code> | const     | contracts/tx-interfaces |
| <code>createContractMaintenanceTxInterface</code> | const     | contracts/tx-interfaces |

**Description** [AI-generated]

These factory functions create the typed call interfaces that wrap circuit calls with the full prove/balance/submit pipeline. The `callTx` object returned by `deployContract` is built from these.

**Verification evidence** [deterministic]

| Test                                               | Description | Expected | Verdict |
|----------------------------------------------------|-------------|----------|---------|
| createCircuitMaintenanceTxInterface is a function  | contracts   | pass     | PASS    |
| createCircuitMaintenanceTxInterfaces is a function | contracts   | pass     | PASS    |
| createContractMaintenanceTxInterface is a function | contracts   | pass     | PASS    |

Example usage [deterministic]

- `createCallTxOptions` — [Example 8: Advanced Deploy Pipeline and Provider Configuration](#), [Example 4: Low-Level Transaction Pipeline](#)
  - `createCircuitCallTxInterface` — [Example 4: Low-Level Transaction Pipeline](#)
  - `createCircuitMaintenanceTxInterface` — [Example 5: Contract Maintenance — Authority and Verifier Keys](#)
  - `createCircuitMaintenanceTxInterfaces` — [Example 5: Contract Maintenance — Authority and Verifier Keys](#)
  - `createContractMaintenanceTxInterface` — [Example 5: Contract Maintenance — Authority and Verifier Keys](#)
- 

### 5.3 Transaction Data Types

Key exports [deterministic]

| Export                                      | Kind              | Source               |
|---------------------------------------------|-------------------|----------------------|
| <code>FinalizedCallTxData</code>            | type              | contracts/tx-model   |
| <code>FinalizedDeployTxData</code>          | type              | contracts/tx-model   |
| <code>FinalizedDeployTxDataBase</code>      | type              | contracts/tx-model   |
| <code>SubmittedCallTx</code>                | type              | contracts/tx-model   |
| <code>UnsubmittedCallTxData</code>          | type              | contracts/tx-model   |
| <code>UnsubmittedDeployTxData</code>        | type              | contracts/tx-model   |
| <code>UnsubmittedDeployTxDataBase</code>    | type              | contracts/tx-model   |
| <code>UnsubmittedDeployTxPrivateData</code> | type              | contracts/tx-model   |
| <code>UnsubmittedDeployTxPublicData</code>  | type              | contracts/tx-model   |
| <code>UnsubmittedTxData</code>              | type              | contracts/tx-model   |
| <code>TxStatus</code>                       | type              | types/midnight-types |
| <code>SucceedEntirely</code>                | const             | types/midnight-types |
| <code>FailFallible</code>                   | const             | types/midnight-types |
| <code>FailEntirely</code>                   | const             | types/midnight-types |
| <code>SegmentStatus</code>                  | type              | types/midnight-types |
| <code>SegmentSuccess</code>                 | const             | types/midnight-types |
| <code>SegmentFail</code>                    | const             | types/midnight-types |
| <code>FinalizedTxData</code>                | interface         | types/midnight-types |
| <code>Fees</code>                           | type              | types/midnight-types |
| <code>BlockHash</code>                      | type              | types/midnight-types |
| <code>Transaction</code>                    | external-reexport | types                |
| <code>UnshieldedBalance</code>              | type              | types/midnight-types |
| <code>UnshieldedBalances</code>             | type              | types/midnight-types |
| <code>UnshieldedUtxo</code>                 | type              | types/midnight-types |
| <code>UnshieldedUtxos</code>                | type              | types/midnight-types |

#### Description [AI-generated]

Transaction data progresses through stages: unsubmitted (local), submitted (in mempool), finalized (on chain). `TxStatus` indicates the outcome: `SucceedEntirely`, `FailFallible` (guaranteed portion succeeded, fallible portion failed), or `FailEntirely`.

#### Verification evidence [deterministic]

| Test                                  | Description | Expected | Verdict |
|---------------------------------------|-------------|----------|---------|
| SegmentFail === "SegmentFail"         | types       | pass     | PASS    |
| SegmentSuccess === "SegmentSuccess"   | types       | pass     | PASS    |
| FailEntirely === "FailEntirely"       | types       | pass     | PASS    |
| FailFallible === "FailFallible"       | types       | pass     | PASS    |
| SucceedEntirely === "SucceedEntirely" | types       | pass     | PASS    |

Example usage [deterministic]

- `SucceedEntirely` — [Example 7: Utility Functions, Type Helpers, and Network Configuration](#)
  - `FailFallible` — [Example 7: Utility Functions, Type Helpers, and Network Configuration](#)
  - `FailEntirely` — [Example 7: Utility Functions, Type Helpers, and Network Configuration](#)
  - `SegmentSuccess` — [Example 7: Utility Functions, Type Helpers, and Network Configuration](#)
  - `SegmentFail` — [Example 7: Utility Functions, Type Helpers, and Network Configuration](#)
  - `Transaction` — [Example 8: Advanced Deploy Pipeline and Provider Configuration](#), [Example 5: Contract Maintenance — Authority and Verifier Keys](#), [Example 4: Low-Level Transaction Pipeline](#), [Example 7: Utility Functions, Type Helpers, and Network Configuration](#)
-

## 6. Error Handling

### Key exports [deterministic]

| Export                                                | Kind  | Package                      |
|-------------------------------------------------------|-------|------------------------------|
| <code>TxFailedError</code>                            | class | contracts                    |
| <code>CallTxFailedError</code>                        | class | contracts                    |
| <code>DeployTxFailedError</code>                      | class | contracts                    |
| <code>ContractTypeError</code>                        | class | contracts                    |
| <code>IncompleteCallTxPrivateStateConfig</code>       | class | contracts                    |
| <code>IncompleteFindContractPrivateStateConfig</code> | class | contracts                    |
| <code>InsertVerifierKeyTxFailedError</code>           | class | contracts                    |
| <code>RemoveVerifierKeyTxFailedError</code>           | class | contracts                    |
| <code>ReplaceMaintenanceAuthorityTxFailedError</code> | class | contracts                    |
| <code>ExportDecryptionError</code>                    | class | types                        |
| <code>ImportConflictError</code>                      | class | types                        |
| <code>InvalidExportFormatError</code>                 | class | types                        |
| <code>InvalidProtocolSchemeError</code>               | class | types                        |
| <code>PrivateStateExportError</code>                  | class | types                        |
| <code>PrivateStateImportError</code>                  | class | types                        |
| <code>PrivateStateImportErrorCause</code>             | type  | types                        |
| <code>SigningKeyExportError</code>                    | class | types                        |
| <code>IndexerFormattedError</code>                    | class | indexer-public-data-provider |

### Description [AI-generated]

The SDK defines a hierarchy of typed errors. Transaction failures (`TxFailedError` and subclasses) carry the transaction status and any partial results. Private state errors handle encryption/decryption failures, import conflicts, and format validation.

### Verification evidence [deterministic]

| Test                                                          | Description | Expected | Verdict |
|---------------------------------------------------------------|-------------|----------|---------|
| MAX_EXPORT_STATES is 10000                                    | types       | pass     | PASS    |
| MAX_EXPORT_SIGNING_KEYS is 10000                              | types       | pass     | PASS    |
| ExportDecryptionError extends Error                           | types       | pass     | PASS    |
| ExportDecryptionError has hardcoded message                   | types       | pass     | PASS    |
| InvalidExportFormatError extends Error                        | types       | pass     | PASS    |
| ImportConflictError stores conflict count                     | types       | pass     | PASS    |
| PrivateStateImportError extends Error                         | types       | pass     | PASS    |
| PrivateStateExportError extends Error                         | types       | pass     | PASS    |
| SigningKeyExportError extends Error                           | types       | pass     | PASS    |
| InvalidProtocolSchemeError stores scheme info                 | types       | pass     | PASS    |
| TxFailedError extends Error                                   | contracts   | pass     | PASS    |
| TxFailedError stores finalized tx data                        | contracts   | pass     | PASS    |
| CallTxFailedError extends Error                               | contracts   | pass     | PASS    |
| CallTxFailedError accepts array of circuit IDs                | contracts   | pass     | PASS    |
| InsertVerifierKeyTxFailedError extends Error                  | contracts   | pass     | PASS    |
| RemoveVerifierKeyTxFailedError extends Error                  | contracts   | pass     | PASS    |
| ReplaceMaintenanceAuthorityTxFailedError extends Error        | contracts   | pass     | PASS    |
| ContractTypeError extends TypeError                           | contracts   | pass     | PASS    |
| IncompleteFindContractPrivateStateConfig extends Error        | contracts   | pass     | PASS    |
| IncompleteFindContractPrivateStateConfig has specific message | contracts   | pass     | PASS    |
| IndexerFormattedError extends Error                           | indexer     | pass     | PASS    |
| IndexerFormattedError stores cause array                      | indexer     | pass     | PASS    |

#### Example usage [deterministic]

- `TxFailedError` — [Example 8: Advanced Deploy Pipeline and Provider Configuration](#), [Example 4: Low-Level Transaction Pipeline](#)
- `CallTxFailedError` — [Example 4: Low-Level Transaction Pipeline](#)
- `DeployTxFailedError` — [Example 4: Low-Level Transaction Pipeline](#)
- `ContractTypeError` — [Example 3: State Observation — Queries, Verification, and Observables](#)
- `IncompleteCallTxPrivateStateConfig` — [Example 8: Advanced Deploy Pipeline and Provider Configuration](#)
- `IncompleteFindContractPrivateStateConfig` — [Example 8: Advanced Deploy Pipeline and Provider Configuration](#)
- `InsertVerifierKeyTxFailedError` — [Example 8: Advanced Deploy Pipeline and Provider Configuration](#)
- `RemoveVerifierKeyTxFailedError` — [Example 8: Advanced Deploy Pipeline and Provider Configuration](#)
- `ReplaceMaintenanceAuthorityTxFailedError` — [Example 8: Advanced Deploy Pipeline and Provider Configuration](#)
- `ExportDecryptionError` — [Example 7: Utility Functions, Type Helpers, and Network Configuration](#)
- `ImportConflictError` — [Example 7: Utility Functions, Type Helpers, and Network Configuration](#)

- `InvalidExportFormatError` — [Example 7: Utility Functions, Type Helpers, and Network Configuration](#)
  - `InvalidProtocolSchemeError` — [Example 7: Utility Functions, Type Helpers, and Network Configuration](#)
  - `PrivateStateExportError` — [Example 7: Utility Functions, Type Helpers, and Network Configuration](#)
  - `PrivateStateImportError` — [Example 7: Utility Functions, Type Helpers, and Network Configuration](#)
  - `SigningKeyExportError` — [Example 7: Utility Functions, Type Helpers, and Network Configuration](#)
  - `IndexerFormattedError` — [Example 8: Advanced Deploy Pipeline and Provider Configuration](#)
-



## 7. Contract Runtime Helpers

Key exports [deterministic]

| Export                                        | Kind  | Package   |
|-----------------------------------------------|-------|-----------|
| <code>ContractProviders</code>                | type  | contracts |
| <code>makeContractExecutableRuntime</code>    | const | types     |
| <code>ContractExecutableRuntimeOptions</code> | type  | types     |
| <code>asContractAddress</code>                | const | types     |
| <code>asEffectOption</code>                   | const | types     |
| <code>exitResultOrError</code>                | const | types     |

Description [AI-generated]

Low-level helpers for contract execution. `makeContractExecutableRuntime` creates the runtime environment that the compiled contract's TypeScript bindings use to execute circuits.

Verification evidence [deterministic]

| Test                                                                           | Description | Expected | Verdict |
|--------------------------------------------------------------------------------|-------------|----------|---------|
| <code>asContractAddress</code> creates branded address                         | types       | pass     | PASS    |
| <code>asEffectOption</code> wraps value                                        | types       | pass     | PASS    |
| <code>asEffectOption</code> handles null/undefined                             | types       | pass     | PASS    |
| <code>exitResultOrError</code> : <code>succeed(42)</code> returns 42           | types       | pass     | PASS    |
| <code>exitResultOrError</code> : <code>succeed("hello")</code> returns "hello" | types       | pass     | PASS    |
| <code>exitResultOrError</code> : <code>fail(Error)</code> throws the error     | types       | pass     | PASS    |
| <code>makeContractExecutableRuntime</code> is a 2-arg function                 | types       | pass     | PASS    |

Example usage [deterministic]

- `makeContractExecutableRuntime` — [Example 8: Advanced Deploy Pipeline and Provider Configuration](#)
- `asContractAddress` — [Example 7: Utility Functions, Type Helpers, and Network Configuration](#)
- `asEffectOption` — [Example 8: Advanced Deploy Pipeline and Provider Configuration](#)
- `exitResultOrError` — [Example 7: Utility Functions, Type Helpers, and Network Configuration](#)

## 8. Network Configuration

Key exports [deterministic]

| Export                    | Kind  | Package    |
|---------------------------|-------|------------|
| <code>setNetworkId</code> | const | network-id |
| <code>getNetworkId</code> | const | network-id |
| <code>NetworkId</code>    | type  | network-id |

Description [AI-generated]

Global network identifier management. `setNetworkId` must be called before any SDK operation to configure which Midnight network (testnet, mainnet) to target.

Example usage [deterministic]

- `setNetworkId` — [Example 8: Advanced Deploy Pipeline and Provider Configuration](#), [Example 5: Contract Maintenance — Authority and Verifier Keys](#), [Example 1: Counter Contract — Deploy, Call, Observe](#), [Example 4: Low-Level Transaction Pipeline](#), [Example 2: Private Counter — Witnesses and Authorization](#), [Example 3: State Observation — Queries, Verification, and Observables](#), [Example 7: Utility Functions, Type Helpers, and Network Configuration](#)
- `getNetworkId` — [Example 8: Advanced Deploy Pipeline and Provider Configuration](#), [Example 7: Utility Functions, Type Helpers, and Network Configuration](#)
- `NetworkId` — [Example 8: Advanced Deploy Pipeline and Provider Configuration](#), [Example 5: Contract Maintenance — Authority and Verifier Keys](#), [Example 1: Counter Contract — Deploy, Call, Observe](#), [Example 4: Low-Level Transaction Pipeline](#), [Example 2: Private Counter — Witnesses and Authorization](#), [Example 3: State Observation — Queries, Verification, and Observables](#)

## 9. Utilities

Key exports [deterministic]

| Export                               | Kind     | Package |
|--------------------------------------|----------|---------|
| <code>toHex</code>                   | const    | utils   |
| <code>fromHex</code>                 | const    | utils   |
| <code>isHex</code>                   | const    | utils   |
| <code>parseHex</code>                | const    | utils   |
| <code>ParsedHexString</code>         | type     | utils   |
| <code>parseCoinPublicKeyToHex</code> | const    | utils   |
| <code>parseEncPublicKeyToHex</code>  | const    | utils   |
| <code>assertIsHex</code>             | function | utils   |
| <code>assertDefined</code>           | function | utils   |
| <code>assertUndefined</code>         | function | utils   |
| <code>assertIsContractAddress</code> | function | utils   |
| <code>ttlOneHour</code>              | const    | utils   |

Description [AI-generated]

Shared utility functions: hex encoding/decoding (Uint8Array <-> hex strings), public key parsing (bech32m -> hex), assertion helpers, and date utilities.

Verification evidence [deterministic]

| Test                                                                          | Description | Expected | Verdict |
|-------------------------------------------------------------------------------|-------------|----------|---------|
| toHex([0xAB, 0xCD]) = "abcd"                                                  | utils       | pass     | PASS    |
| toHex([]) = ""                                                                | utils       | pass     | PASS    |
| toHex([0x00, 0xFF]) = "00ff"                                                  | utils       | pass     | PASS    |
| toHex single byte [0x0A] = "0a"                                               | utils       | pass     | PASS    |
| fromHex("abcd") → [0xAB, 0xCD]                                                | utils       | pass     | PASS    |
| fromHex("ABCD") → [0xAB, 0xCD] (case insensitive)                             | utils       | pass     | PASS    |
| fromHex("") → empty buffer                                                    | utils       | pass     | PASS    |
| fromHex roundtrip: fromHex(toHex(bytes)) === bytes                            | utils       | pass     | PASS    |
| isHex("abcd") = true                                                          | utils       | pass     | PASS    |
| isHex("ABCD") = true (case insensitive)                                       | utils       | pass     | PASS    |
| isHex("abcd", 2) = true (2 bytes)                                             | utils       | pass     | PASS    |
| isHex("abcd", 3) = false (wrong byte length)                                  | utils       | pass     | PASS    |
| isHex("") = false (empty)                                                     | utils       | pass     | PASS    |
| isHex("xyz") = false (non-hex)                                                | utils       | pass     | PASS    |
| isHex("abc") = false (odd length)                                             | utils       | pass     | PASS    |
| isHex("0xabcd") — check prefix handling                                       | utils       | pass     | PASS    |
| parseHex("abcd") = {hasPrefix: false, byteChars: "abcd", incompleteChars: ""} | utils       | pass     | PASS    |
| parseHex("0xabcd") = {hasPrefix: true, byteChars: "abcd", ...}                | utils       | pass     | PASS    |
| parseHex("abc") — odd length has incompleteChars                              | utils       | pass     | PASS    |
| parseHex("abgh") — non-hex has incompleteChars                                | utils       | pass     | PASS    |
| assertIsHex("abcd") — valid, no throw                                         | utils       | pass     | PASS    |
| assertIsHex("abcd", 2) — valid length, no throw                               | utils       | pass     | PASS    |
| assertIsHex("") — empty throws                                                | utils       | pass     | PASS    |
| assertIsHex("xyz") — non-hex throws                                           | utils       | pass     | PASS    |
| assertIsHex("abcd", 3) — wrong length throws                                  | utils       | pass     | PASS    |
| assertDefined(42) — no throw                                                  | utils       | pass     | PASS    |
| assertDefined("hello") — no throw                                             | utils       | pass     | PASS    |
| assertDefined(null) — throws                                                  | utils       | pass     | PASS    |
| assertDefined(undefined) — throws                                             | utils       | pass     | PASS    |
| assertDefined(null, "custom msg") — throws with message                       | utils       | pass     | PASS    |
| assertUndefined(null) — no throw                                              | utils       | pass     | PASS    |
| assertUndefined(undefined) — no throw                                         | utils       | pass     | PASS    |
| assertUndefined(42) — throws                                                  | utils       | pass     | PASS    |

| Test                                                                                                                                          | Description | Expected | Verdict |
|-----------------------------------------------------------------------------------------------------------------------------------------------|-------------|----------|---------|
| <code>assertIsContractAddress("aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa")</code><br>— valid, no throw | utils       | pass     | PASS    |
| <code>assertIsContractAddress("0x" + "a"*64)</code> — 0x prefix rejected                                                                      | utils       | pass     | PASS    |
| <code>assertIsContractAddress("short")</code> — too short throws                                                                              | utils       | pass     | PASS    |
| <code>assertIsContractAddress("a"*62)</code> — 31 bytes throws                                                                                | utils       | pass     | PASS    |
| <code>parseCoinPublicKeyToHex</code> passes through valid 64-char hex string                                                                  | utils       | pass     | PASS    |
| <code>parseCoinPublicKeyToHex</code> passes through uppercase hex                                                                             | utils       | pass     | PASS    |
| <code>parseCoinPublicKeyToHex</code> : hex passthrough skips bech32m decoding                                                                 | utils       | pass     | PASS    |
| <code>parseCoinPublicKeyToHex</code> rejects non-hex non-bech32m input                                                                        | utils       | pass     | PASS    |
| bech32m roundtrip: encode 32 bytes as bech32m, decode back to hex                                                                             | utils       | pass     | PASS    |
| bech32m: encoded string starts with <code>mn_</code> prefix                                                                                   | utils       | pass     | PASS    |
| bech32m: encoded string contains shield-cpk type segment                                                                                      | utils       | pass     | PASS    |
| bech32m: different keys produce different bech32m strings                                                                                     | utils       | pass     | PASS    |
| bech32m: <code>parseCoinPublicKeyToHex</code> rejects bech32m with wrong network                                                              | utils       | pass     | PASS    |
| bech32m: hex strings still pass through even with real utils                                                                                  | utils       | pass     | PASS    |
| <code>parseEncPublicKeyToHex</code> passes through valid hex string                                                                           | utils       | pass     | PASS    |
| <code>parseCoinPublicKeyToHex</code> : hex input passes through                                                                               | utils       | pass     | PASS    |
| <code>parseCoinPublicKeyToHex</code> : uppercase hex passes through                                                                           | utils       | pass     | PASS    |
| <code>parseCoinPublicKeyToHex</code> : non-hex non-bech32m throws                                                                             | utils       | pass     | PASS    |
| <code>parseEncPublicKeyToHex</code> : hex input passes through                                                                                | utils       | pass     | PASS    |
| <code>parseEncPublicKeyToHex</code> : non-hex non-bech32m throws                                                                              | utils       | pass     | PASS    |

### Example usage [deterministic]

- `toHex` — [Example 4: Low-Level Transaction Pipeline](#), [Example 7: Utility Functions, Type Helpers, and Network Configuration](#)
- `fromHex` — [Example 4: Low-Level Transaction Pipeline](#), [Example 7: Utility Functions, Type Helpers, and Network Configuration](#)
- `isHex` — [Example 4: Low-Level Transaction Pipeline](#), [Example 7: Utility Functions, Type Helpers, and Network Configuration](#)
- `parseHex` — [Example 7: Utility Functions, Type Helpers, and Network Configuration](#)
- `parseCoinPublicKeyToHex` — [Example 8: Advanced Deploy Pipeline and Provider Configuration](#), [Example 7: Utility Functions, Type Helpers, and Network Configuration](#)
- `parseEncPublicKeyToHex` — [Example 7: Utility Functions, Type Helpers, and Network Configuration](#)
- `assertIsHex` — [Example 7: Utility Functions, Type Helpers, and Network Configuration](#)
- `assertDefined` — [Example 7: Utility Functions, Type Helpers, and Network Configuration](#)
- `assertUndefined` — [Example 7: Utility Functions, Type Helpers, and Network Configuration](#)
- `assertIsContractAddress` — [Example 4: Low-Level Transaction Pipeline](#), [Example 7: Utility Functions, Type Helpers, and Network Configuration](#)

- `ttlOneHour` — [Example 7: Utility Functions, Type Helpers, and Network Configuration](#)
-

## 10. Full-Flow Integration Tests

Description

[AI-generated]

The end-to-end transaction pipeline, tested against a local devnet with all infrastructure running (node, indexer, proof server). These tests exercise the complete SDK lifecycle:

- 1. **Key derivation** — HD wallet seed → ZSwap, unshielded, and dust keys
- 2. **WalletFacade initialization** — shielded + unshielded + dust wallets
- 3. **Provider assembly** — ZK config, proof server, indexer, private state
- 4. **Contract loading** — compiled counter contract via CompiledContract builder
- 5. **Local execution** — constructor and circuit execution without blockchain
- 6. **Infrastructure integration** — proof server /check, indexer queries
- 7. **Deployment** — full `deployContract` pipeline (prove → balance → submit → watch)

Prerequisites: local devnet running ( `docker compose up -d` ), counter contract self-compiled ( `bash workspace/compile-contracts.sh` ).

**Note:** Full-flow evidence requires a running devnet (Tier 3) and a self-compiled counter contract. The compilation step is included in `run-tests.sh --devnet`. Without devnet, this section has no evidence.

Verification evidence

[deterministic]

| Test | Description | Expected | Verdict |
|------|-------------|----------|---------|
|------|-------------|----------|---------|

## 11. End-to-End Examples

Working examples that exercise the complete SDK lifecycle against a local devnet. Each example produces structured JSON evidence via `--json`, suitable for deterministic verification.

Prerequisites for all examples:

- Devnet running: `docker compose up -d node indexer proof-server`
- Services healthy: `bash scripts/wait-for-devnet.sh`
- Dependencies installed: `cd workspace && npm install`
- Contracts compiled: `bash workspace/compile-contracts.sh`
- Node.js  $\geq 22$  (wallet SDK requires ES2024 iterator helpers)

### Example 1: Counter Contract — Deploy, Call, Observe

This example walks through the complete lifecycle of a Midnight smart contract: deploying it, calling its circuits, and observing state changes on the blockchain. Every claim is backed by evidence from a live devnet run.

Source: `workspace/examples/counter-e2e.mjs`

#### The contract

The counter contract source (self-compiled from `workspace/test-contracts/counter.compact`):

```
import CompactStandardLibrary;

export ledger round: Counter;

export circuit increment(): [] {
  round.increment(1);
}
```

**[AI-generated]** This is the simplest possible Midnight smart contract — the "Hello World" of zero-knowledge DApps. It declares:

- **One ledger field** (`round`) — a `Counter` that lives on-chain, visible to everyone, starting at 0
- **One circuit** (`increment`) — adds 1 to the counter

This contract is compiled from our own Compact source using the `compactc` compiler (v0.30.0), not taken from upstream pre-compiled binaries. The compilation pipeline (`workspace/compile-contracts.sh`) produces the contract TypeScript bindings, ZKIR circuits, and prover/verifier keys — closing the gap between Compact source and deployed contract.

When a user calls `increment()`, the SDK:

1. Executes the circuit locally to compute the new state
2. Sends the computation to the proof server, which generates a ZK proof
3. Packages the proof into a transaction with fee inputs
4. Submits the transaction to the Midnight node
5. The blockchain verifies the proof and updates the ledger state

The user never reveals *how* they computed the new state — only that the state transition is valid according to the circuit's constraints.



Step 1: Wait for wallet sync

[AI-generated]

After calling `walletFacade.start()` (see Section 5), the three sub-wallets begin syncing with the indexer in the background. The dust wallet must discover its fee tokens before any transaction can be balanced. If you attempt to deploy or call a circuit before sync completes, you'll get `No dust found in the wallet state`.

The fix: subscribe to `walletFacade.state()` (an RxJS Observable) and wait until all three sub-wallets report their sync progress as complete.

**Source** — extracted from `counter-e2e.mjs`:

```
await new Promise((resolve, reject) => {
  let updateCount = 0;
  const timeout = setTimeout(() => {
    sub.unsubscribe();
    reject(new Error('Wallet sync timeout after 90s'));
  }, 90_000);
  const sub = walletFacade.state().subscribe({
    next: (state) => {
      updateCount++;
      const shielded = state.shielded.state.progress.isStrictlyComplete();
      const dust = state.dust.state.progress.isStrictlyComplete();
      const unshielded = state.unshielded.progress.isStrictlyComplete();
      if (verbose && (updateCount <= 5 || updateCount % 10 === 0)) {
        console.log(` [sync #${updateCount}] shielded=${shielded} dust=${dust} unshielded=${unshielded}`);
      }
      if (shielded && dust && unshielded) {
        console.log(` [sync] All wallets synced after ${updateCount} updates`);
        clearTimeout(timeout);
        sub.unsubscribe();
        resolve();
      }
    },
    error: (err) => {
      clearTimeout(timeout);
      reject(err);
    },
  });
});
```

[deterministic]

| Step                                             | Detail                                       | Verdict |
|--------------------------------------------------|----------------------------------------------|---------|
| WalletFacade initialized and synced              | ShieldedWallet, UnshieldedWallet, DustWallet | PASS    |
| WalletFacade initialized and synced with indexer | ShieldedWallet, UnshieldedWallet, DustWallet | PASS    |
| WalletFacade initialized and synced              | ShieldedWallet, UnshieldedWallet, DustWallet | PASS    |
| WalletFacade initialized and synced              | ShieldedWallet, UnshieldedWallet, DustWallet | PASS    |

Step 2: Deploy the contract

**[AI-generated]** `deployContract()` handles the full deploy pipeline internally: it creates an unproven deploy transaction, sends it to the proof server to generate a ZK proof, passes it to the wallet for fee balancing, submits it to the node, and watches the indexer for confirmation. One call does everything.

**Source** — extracted from `counter-e2e.mjs`:

```
const deployed = await deployContract(providers, {
  compiledContract,
  privateStateId: `counter-deploy-${Date.now()}`,
  initialPrivateState: {},
});

const contractAddress = deployed.deployTxData.public.contractAddress;
const deployTxId = deployed.deployTxData.public.txId;
const deployBlock = deployed.deployTxData.public.blockHeight;
```

**[deterministic]**

| Step                        | Detail                                                                                               | Verdict |
|-----------------------------|------------------------------------------------------------------------------------------------------|---------|
| Counter contract deployed   | tx: <code>00b095e51950e3d6...</code> , block: 6733, 19.7s, address: <code>864eb8f62d361454...</code> | PASS    |
| Contract deployed to devnet | tx: <code>006f1699d7498240...</code> , block: 5024, 20.7s, address: <code>62c7c2574f799871...</code> | PASS    |

Step 3: Read the initial counter value

**[AI-generated]** After deploying, the contract's ledger state is visible on the blockchain via the indexer. The `ledger()` function is generated by the Compact compiler alongside the Contract class — it decodes the raw on-chain bytes into typed JavaScript values.

**Source** — extracted from `counter-e2e.mjs`:

```
const readState = async (label) => {
  const state = await publicDataProvider.queryContractState(contractAddress);
  if (!state) throw new Error('Contract state not found on indexer');
  const decoded = counterLedger(state.data);
  const counterValue = decoded.round; // → bigint
  return counterValue;
};
```

**[deterministic]**

| Step                          | Detail  | Verdict |
|-------------------------------|---------|---------|
| Counter value after deploy: 0 | round=0 | PASS    |
| Counter value after deploy: 0 | round=0 | PASS    |

### Step 4: Call the increment circuit

**[AI-generated]** `callTx.increment()` triggers the full proof pipeline: execute the circuit locally, generate a ZK proof via the proof server, balance the transaction (add fee inputs), submit to the node, and wait for indexer confirmation. The counter value increases by exactly 1.

**Source** — extracted from `counter-e2e.mjs`:

```
const inc1Result = await deployed.callTx.increment();
const valueAfterInc1 = await readState('after first increment');
```

**[deterministic]**

| Step                                 | Detail                                                           | Verdict |
|--------------------------------------|------------------------------------------------------------------|---------|
| Increment #1: counter 0 -> 1         | tx: <code>00020d3564362a1f...</code> , 0 → 1                     | PASS    |
| Increment #1: counter 0 -> 1         | tx: <code>00811b2111af521c...</code> , block: 5027, 18.5s, 0 → 1 | PASS    |
| Private counter increment #1: 0 -> 1 | tx: <code>008b6f0829732286...</code> , block: 5040, 17.1s, 0 → 1 | PASS    |

### Step 5: Increment again **[AI-generated]**

State persists across transactions. A second increment proves the counter correctly advances from 1 to 2.

**[deterministic]**

| Step                                      | Detail                                                           | Verdict |
|-------------------------------------------|------------------------------------------------------------------|---------|
| Increment #2 (re-enabled): counter 1 -> 2 | tx: <code>0090376328154333...</code> , 1 → 2                     | PASS    |
| Increment #2: counter 1 -> 2              | tx: <code>00f9015ce2fc11d8...</code> , block: 5030, 17.3s, 1 → 2 | PASS    |
| Private counter increment #2: 1 -> 2      | tx: <code>0007bff7a06c8edb...</code> , block: 5043, 18.7s, 1 → 2 | PASS    |

### Step 6: Reconnect to the deployed contract

**[AI-generated]** In a real DApp, the user may close the browser and come back later. `findDeployedContract()` reconnects to an already-deployed contract using only its on-chain address — no need to redeploy.

**Source** — extracted from `counter-e2e.mjs`:

```
const found = await findDeployedContract(
  { ...providers, privateStateProvider: freshPrivateStateProvider },
  {
    compiledContract,
    contractAddress,
    privateStateId: `example-counter-reconnect-${Date.now()}`,
    initialPrivateState: {},
  },
);

const reconnectedAddress = found.deployTxData.public.contractAddress;
```

[deterministic]

| Step                             | Detail                       | Verdict |
|----------------------------------|------------------------------|---------|
| Reconnected to deployed contract | address: 62C7c2574f799871... | PASS    |

Step 7: Increment via reconnected client [AI-generated]

The reconnected client has full functionality — it can call circuits and observe state changes just like the original deployer.

[deterministic]

| Step                                       | Detail                                             | Verdict |
|--------------------------------------------|----------------------------------------------------|---------|
| Increment #3 (final): counter 2 -> 3       | tx: 009796b529742593..., 2 → 3                     | PASS    |
| Increment #3 (reconnected): counter 2 -> 3 | tx: 00f3642060a16a36..., block: 5033, 18.3s, 2 → 3 | PASS    |

Verification [AI-generated]

The devnet is the oracle for this example. For each step:

- **Deploy/Increment:** the node accepted the transaction and included it in a block — this is only possible if the proof server generated a valid ZK proof
- **State query:** the indexer returned the decoded counter value, confirming the on-chain state changed as expected
- **Reconnect:** the SDK found the contract at the same address and reconstructed a working client

If the proof were invalid, the node would reject the transaction. If the state transition violated the circuit's constraints, the proof server would fail to generate a proof.

The complete source is at workspace/examples/counter-e2e.mjs. Run with --verbose for detailed output at each step.

Example 2: Private Counter — Witnesses and Authorization

This example builds on the counter by adding a **private witness**: a secret key that authorizes who can increment the counter. The zero-knowledge proof guarantees that the caller knows the secret key without revealing it on-chain.

Source: workspace/examples/private-counter-e2e.mjs

The contract

The private counter contract source (self-compiled from workspace/test-contracts/private-counter.compact):

```
import CompactStandardLibrary;

export ledger count: Counter;
export ledger owner: Bytes<32>;

witness secret_key(): Bytes<32>;

circuit owner_hash(sk: Bytes<32>): Bytes<32> {
  return disclose(persistentHash<Vector<2, Bytes<32>>>([pad(32, "owner:"), sk]));
}

constructor() {
  owner = owner_hash(secret_key());
}

export circuit increment(): Uint<32> {
  assert(owner_hash(secret_key()) == owner, "not the owner");
  count.increment(1);
  return count as Uint<32>;
}
```

**[AI-generated]** This contract extends the basic counter pattern with **key-based authorization**:

- **count (ledger)** — the counter, visible on-chain, starting at 0
- **owner (ledger)** — stores a hash of the secret key, visible on-chain but not reversible
- **secret\_key (witness)** — the private input, never revealed on-chain
- **owner\_hash (helper circuit)** — computes a deterministic hash of the key using `persistentHash`
- **constructor** — sets the owner to `owner_hash(secret_key())` at deploy time
- **increment** — asserts that `owner_hash(secret_key()) == owner` before incrementing; returns the new count

The privacy guarantee: the zero-knowledge proof proves the caller *knows* a key whose hash matches `owner`, without revealing the key itself. The node verifies the proof but never sees `secret_key`.

### Step 1: Set up witnesses **[AI-generated]**

Unlike the basic counter, this contract requires a **witness function** that provides the secret key during circuit execution. The witness runs locally — it's never sent to the network.

**Source** — extracted from `private-counter-e2e.mjs`:

```
// The secret key – a 32-byte value known only to the deployer.
// In production, this would come from a wallet or key store.
const SECRET_KEY = new Uint8Array(32);
SECRET_KEY.set(new TextEncoder().encode('aletheia-demo-secret'));

// Witness function: called by the contract during circuit execution.
// The runtime calls secret_key(context) and expects back a tuple:
// [nextPrivateState, result]
// - nextPrivateState: updated private state (pass through unchanged)
// - result: the actual Bytes<32> value
// This runs locally – the key never leaves the client machine.
const witnesses = {
  secret_key: (ctx) => [ctx.privateState, SECRET_KEY],
};
```

Step 2: Deploy with constructor [AI-generated]

The constructor circuit runs during deployment, calling `secret_key()` and storing `owner_hash(secret_key())` on-chain. The proof guarantees the owner field was set correctly without revealing the key.

Source — extracted from `private-counter-e2e.mjs`:

```
const deployed = await deployContract(providers, {
  compiledContract,
  privateStateId: `private-counter-deploy-${Date.now()}`,
  initialPrivateState: {},
});

const contractAddress = deployed.deployTxData.public.contractAddress;
const deployTxId = deployed.deployTxData.public.txId;
const deployBlock = deployed.deployTxData.public.blockHeight;
```

[deterministic]

| Step                               | Detail                                                                   | Verdict |
|------------------------------------|--------------------------------------------------------------------------|---------|
| Private counter deployed to devnet | tx: 000f42030dde306..., block: 5037, 21.1s, address: 241ec3f8a52adde3... | PASS    |

Step 3: Verify the privacy property

[AI-generated] The on-chain `owner` field contains a hash — a 32-byte value derived from the secret key via `persistentHash`. We verify it is NOT the raw key bytes, confirming that the key remains private.

Source — extracted from `private-counter-e2e.mjs`:

```
const readState = async () => {
  const state = await publicDataProvider.queryContractState(contractAddress);
  if (!state) throw new Error('Contract state not found on indexer');
  const decoded = pcLedger(state.data);
  return {
    count: decoded.count,    // Counter → bigint
    owner: decoded.owner,    // Bytes<32> → hex string or Uint8Array
  };
};
```

[deterministic]

| Step                                           | Detail | Verdict |
|------------------------------------------------|--------|---------|
| Owner field is hash of secret key, not raw key |        | PASS    |

Step 4: Increment with authorization [AI-generated]

Each call to `increment()` re-proves knowledge of the secret key. The proof server generates a ZK proof that the caller knows a key whose hash matches `owner`, and that the count increased by 1.

Source — extracted from `private-counter-e2e.mjs`:

```
const inc1Result = await deployed.callTx.increment();
const stateAfterInc1 = await readState();
const valueAfterInc1 = stateAfterInc1.count;
```

[deterministic]

| Step                                 | Detail                                                           | Verdict |
|--------------------------------------|------------------------------------------------------------------|---------|
| Private counter increment #1: 0 -> 1 | tx: <code>008b6f0829732286...</code> , block: 5040, 17.1s, 0 → 1 | PASS    |

Step 5: Increment again [AI-generated]

A second increment confirms state persistence across authorized transactions.

[deterministic]

| Step                                 | Detail                                                            | Verdict |
|--------------------------------------|-------------------------------------------------------------------|---------|
| Private counter increment #2: 1 -> 2 | tx: <code>0007bfff7a06c8edb...</code> , block: 5043, 18.7s, 1 → 2 | PASS    |

Verification [AI-generated]

This example demonstrates three properties:

- **Witnesses are private** — the secret key never appears on-chain; only its hash is stored in the `owner` ledger field
- **Authorization works** — the `assert` in `increment()` is enforced by the ZK proof; a caller who doesn't know the secret key cannot generate a valid proof

- **State transitions are correct** — each authorized increment adds exactly 1

The proof server is the oracle: if the proof generation succeeds and the node accepts the transaction, the circuit constraints are satisfied.

The complete source is at `workspace/examples/private-counter-e2e.mjs`.

---

## Example 3: State Observation — Queries, Verification, and Observables

This example demonstrates the different ways to **read and verify** contract state in MidnightJS. Where Examples 1 and 2 focused on deploying and calling circuits, this example focuses on what happens *between* transactions: querying state, verifying contract integrity, and subscribing to real-time state changes.

Source: `workspace/examples/state-observation-e2e.mjs`

### The contract

This example reuses the counter contract from Example 1 — the focus is on observation, not on the contract itself.

### Step 1: Verify contract integrity [AI-generated]

After deploying a contract, you can verify that the on-chain verifier keys match your local compiled contract. This is critical for DApp security: it ensures you're interacting with the contract you think you are, not a different contract at the same address.

`verifierKeysEqual` performs a byte-level comparison of two verifier keys. `verifyContractState` checks *all* circuits at once — it throws `ContractTypeError` if any circuit's on-chain key doesn't match.

**Source** — extracted from `state-observation-e2e.mjs`:



```
// Read the on-chain contract state
const contractState = await publicDataProvider.queryContractState(contractAddress);

// Load the local verifier key for comparison
const localVerifierKey = await zkConfigProvider.getVerifierKey('increment');

// verifierKeysEqual: byte-level comparison of two verifier keys.
// Returns true if they are identical – used to detect contract tampering
// or version mismatches.
const keysMatch = verifierKeysEqual(localVerifierKey, localVerifierKey);

// verifyContractState: checks that the on-chain state contains verifier
// keys matching the local compiled contract. Throws ContractTypeError
// if any circuit's keys don't match.
// We pass an array of [circuitId, verifierKey] tuples.
let verifyPassed = false;
try {
  verifyContractState(
    [['increment', localVerifierKey]],
    contractState,
  );
  verifyPassed = true;
} catch (err) {
  verifyPassed = false;
}
```

[deterministic]

| Step                                             | Detail | Verdict |
|--------------------------------------------------|--------|---------|
| verifyContractState passes for deployed contract |        | PASS    |

Step 2: Read state via `getPublicStates` [AI-generated]

`getPublicStates` is the simplest way to read on-chain state. It returns both the ZSwap chain state (used internally by the wallet) and the contract's ledger state, without requiring a private state provider.

Source — extracted from `state-observation-e2e.mjs`:

```
// getPublicStates: fetches the on-chain state for a contract address.
// Returns { zswapChainState, contractState } – no private state needed.
const publicStates = await getPublicStates(publicDataProvider, contractAddress);

// Decode the contract state using the compiler-generated ledger decoder
const decodedPublic = counterLedger(publicStates.contractState.data);
const publicRound = decodedPublic.round; // → bigint
```

[deterministic]

| Step                       | Detail | Verdict |
|----------------------------|--------|---------|
| getPublicStates: round = 0 |        | PASS    |

Step 3: Read state via getStates (public + private) [AI-generated]

`getStates` combines public state (from the indexer) with private state (from the local encrypted store). This is the full state view that a DApp typically needs — the on-chain data plus the user's local secrets.

Source — extracted from `state-observation-e2e.mjs`:

```
// getStates: fetches public state from the indexer AND private state
// from the local store. Returns { zswapChainState, contractState, privateState }.
const fullStates = await getStates(
  publicDataProvider,
  privateStateProvider,
  contractAddress,
  privateStateId,
);

const decodedFull = counterLedger(fullStates.contractState.data);
const fullRound = decodedFull.round;
const hasPrivateState = fullStates.privateState !== undefined;
```

[deterministic]

| Step                                         | Detail | Verdict |
|----------------------------------------------|--------|---------|
| getStates: round = 0, hasPrivateState = true |        | PASS    |

Step 4: Query unshielded balances [AI-generated]

`getUnshieldedBalances` queries token balances at a contract address. A pure counter contract has no token operations, so the balance is empty — but the function still works. In a token contract, this would return the unshielded token amounts.

Source — extracted from `state-observation-e2e.mjs`:

```
// getUnshieldedBalances: queries unshielded token balances at a contract
// address. Returns a Map of token type → balance.
// A pure counter contract has no token operations, so balances are empty.
const balances = await getUnshieldedBalances(publicDataProvider, contractAddress);
const balanceCount = balances ? (balances.size ?? Object.keys(balances).length) : 0;
```

[deterministic]

| Step                                 | Detail | Verdict |
|--------------------------------------|--------|---------|
| getUnshieldedBalances: 0 token types |        | PASS    |

## Step 5: Subscribe to state changes via observable [AI-generated]

`contractStateObservable` returns an RxJS Observable that emits whenever the contract's on-chain state changes. This enables reactive UIs — instead of polling, your DApp receives push notifications of state changes.

Configuration options control the starting point: `'latest'` for the most recent state, `'all'` for the full history, or a specific transaction/block.

**Source** — extracted from `state-observation-e2e.mjs`:

```
// contractStateObservable: returns an Observable<ContractState> that
// emits whenever the contract's on-chain state changes.
// Config options: { type: 'all' }, { type: 'latest' },
// { type: 'txId', txId }, { type: 'blockHeight', blockHeight },
// { type: 'blockHash', blockHash } – each with optional `inclusive`.
//
// Here we subscribe from 'latest' to catch the next state change.
const observedStates = [];
const observablePromise = new Promise((resolve, reject) => {
  const timeout = setTimeout(() => {
    sub.unsubscribe();
    resolve(observedStates); // resolve with whatever we got
  }, 60_000);

  const sub = publicDataProvider.contractStateObservable(
    contractAddress,
    { type: 'latest' },
  ).subscribe({
    next: (state) => {
      observedStates.push(state);
      if (verbose) {
        const decoded = counterLedger(state.data);
        console.log(` [observable] State update: round = ${decoded.round}`);
      }
      // We expect at least one state emission after increment
      if (observedStates.length >= 1) {
        clearTimeout(timeout);
        sub.unsubscribe();
        resolve(observedStates);
      }
    },
    error: (err) => {
      clearTimeout(timeout);
      reject(err);
    },
  });
});

// Increment the counter – this will trigger the observable
console.log(' Incrementing counter to trigger observable...');
const incStart = Date.now();
const incResult = await deployed.callTx.increment();
const incMs = Date.now() - incStart;

// Wait for the observable to receive the state change
const observed = await observablePromise;
```

[deterministic]

| Step                                  | Detail | Verdict |
|---------------------------------------|--------|---------|
| Observable received 1 state update(s) |        | PASS    |

Verification [AI-generated]

This example demonstrates five ways to interact with contract state:

- **verifyContractState** — proves the on-chain contract matches the local compiled code
- **getPublicStates** — one-shot public state query (no private state)
- **getStates** — combined public + private state query
- **getUnshieldedBalances** — token balance query
- **contractStateObservable** — reactive state subscription

Together, these cover the full spectrum from simple reads to real-time subscriptions. The devnet is the oracle: if the queries return valid data and the observable fires, the APIs work as documented.

The complete source is at `workspace/examples/state-observation-e2e.mjs`.

Example 4: Low-Level Transaction Pipeline

Examples 1-3 used the high-level `deployContract` and `callTx` APIs, which wrap the entire prove → balance → submit → watch pipeline in one call. This example peels back the layers to show the individual steps, giving you control over each stage of the transaction lifecycle.

Source: `workspace/examples/low-level-pipeline-e2e.mjs`

The contract

This example reuses the counter contract — the focus is on the transaction pipeline, not on the contract.

Step 1: Import low-level functions [AI-generated]

The SDK exports both high-level convenience functions and the individual pipeline stages they're built from. The low-level functions give you control over each step, while the error classes enable typed error handling.

Source — extracted from `low-level-pipeline-e2e.mjs`:

```
// Low-level contracts API – the individual pipeline stages
const {
  // High-level (used for initial deploy only)
  deployContract,
  // Low-level call tx creation
  createUnprovenCallTx,
  createCallTxOptions,
  // Mid-level call submission
  submitCallTx,
  submitCallTxAsync,
  // Generic tx submission (prove → balance → submit → watch)
  submitTx,
  // Interface builders
  createCircuitCallTxInterface,
  // Error hierarchy
  TxFailedError,
  CallTxFailedError,
  DeployTxFailedError,
} = await wsImport('@midnight-ntwrk/midnight-js-contracts/dist/index.mjs');

// Utility functions for hex encoding and address validation
const {
  toHex,
  fromHex,
  isHex,
  assertIsContractAddress,
} = await wsImport('@midnight-ntwrk/midnight-js-utils/dist/index.mjs');
```

## Step 2: Utility functions [AI-generated]

The SDK provides hex encoding utilities ( `toHex` , `fromHex` , `isHex` ) and assertion helpers ( `assertIsContractAddress` ) used throughout the pipeline. These handle the conversion between Uint8Array (used internally) and hex strings (used for display and addresses).

**Source** — extracted from `low-level-pipeline-e2e.mjs`:

```
// toHex: Uint8Array → hex string
const coinKeyHex = toHex(zswapSecretKeys.coinPublicKey);

// fromHex: hex string → Uint8Array
const coinKeyBytes = fromHex(coinKeyHex);
const roundTrip = toHex(coinKeyBytes) === coinKeyHex;

// isHex: validate a hex string
const addressIsHex = isHex(contractAddress);
const invalidHex = isHex('not-a-hex-string');

// assertIsContractAddress: throws if the string is not a valid address
let addressValid = false;
try {
  assertIsContractAddress(contractAddress);
  addressValid = true;
} catch {
  addressValid = false;
}
```

[deterministic]

| Step                                    | Detail | Verdict |
|-----------------------------------------|--------|---------|
| Hex utilities: toHex/fromHex round-trip |        | PASS    |

Step 3: Error class hierarchy [AI-generated]

The SDK defines a typed error hierarchy for transaction failures. `TxFailedError` is the base class; `CallTxFailedError` and `DeployTxFailedError` extend it with context-specific data (partial results, circuit output, deploy tx data). This enables precise `catch` blocks in production DApps.

Source — extracted from `low-level-pipeline-e2e.mjs`:

```
// TxFailedError is the base class for all transaction failures.
// CallTxFailedError and DeployTxFailedError extend it with
// context-specific details (circuit results, deploy data, etc.).
const callTxError = new CallTxFailedError('test');
const deployTxError = new DeployTxFailedError('test');

const callIsInstanceOfBase = callTxError instanceof TxFailedError;
const deployIsInstanceOfBase = deployTxError instanceof TxFailedError;
const callIsInstanceOfCall = callTxError instanceof CallTxFailedError;
const deployIsInstanceOfDeploy = deployTxError instanceof DeployTxFailedError;
```

[deterministic]

| Step                                                       | Detail | Verdict |
|------------------------------------------------------------|--------|---------|
| Error hierarchy: CallTxFailedError extends TxFailedError   |        | PASS    |
| Error hierarchy: DeployTxFailedError extends TxFailedError |        | PASS    |

Step 4: Manual call pipeline [AI-generated]

This is what `callTx.increment()` does internally, broken into visible steps:

- 1. `createCallTxOptions` — builds the typed options object from compiled contract, circuit ID, address, and arguments
- 2. `createUnprovenCallTx` — executes the circuit locally and produces proof data (no network call yet)
- 3. `submitTx` — takes the unproven transaction through the full pipeline: prove → balance → submit → watch

Doing it manually gives you control over each stage — useful for custom balancing logic, detailed logging, or retry strategies.

Source — extracted from `low-level-pipeline-e2e.mjs`:

```
// Step 8a: Build call options using the helper.
// createCallTxOptions constructs the typed options object:
// (compiledContract, circuitId, contractAddress, privateStateId, args)
const callOptions = createCallTxOptions(
  compiledContract,    // the compiled contract descriptor
  'increment',         // circuit to call
  contractAddress,     // on-chain contract address
  privateStateId,      // private state store key
  [],                  // circuit arguments (increment takes none)
);

// Step 8b: Create the unproven transaction.
// This executes the circuit LOCALLY – no network call yet.
// It produces proof data that the proof server will turn into a ZK proof.
console.log('  Creating unproven call transaction...');
const unprovenCallData = await createUnprovenCallTx(providers, callOptions);

// The unproven data contains the circuit execution result:
const unprovenTx = unprovenCallData.private.unprovenTx;

// Step 8c: Submit the transaction through the full pipeline.
// submitTx does: prove → balance → submit → watch for confirmation.
console.log('  Submitting: prove → balance → submit → watch...');
const finalizedTxData = await submitTx(providers, {
  unprovenTx,
  circuitId: 'increment',
});
```



[deterministic]

| Step                        | Detail                                       | Verdict |
|-----------------------------|----------------------------------------------|---------|
| Low-level increment: 0 -> 1 | tx: 00684a284e45948d... , block: 5057, 17.0s | PASS    |

Step 5: Build callTx interface from scratch [AI-generated]

`deployContract` returns `{ callTx: { increment: fn } }`. That `callTx` object is built internally by `createCircuitCallTxInterface`. Here we reconstruct it from scratch — useful if you need to build a call interface from a contract address and compiled contract without going through `findDeployedContract`.

Source — extracted from `low-level-pipeline-e2e.mjs`:

```
// createCircuitCallTxInterface: builds the typed call interface
// from providers, compiled contract, address, and private state ID.
// This is what deployContract and findDeployedContract use internally.
const manualCallTx = createCircuitCallTxInterface(
  providers,
  compiledContract,
  contractAddress,
  privateStateId,
);

// The result has one method per impure circuit.
// For the counter contract, that's just `increment()`.
const hasIncrement = typeof manualCallTx.increment === 'function';
```

[deterministic]

| Step                            | Detail                                       | Verdict |
|---------------------------------|----------------------------------------------|---------|
| Manual callTx interface: 1 -> 2 | tx: 00a6997649d2bb73... , block: 5060, 18.7s | PASS    |

Step 6: Async submission [AI-generated]

`submitCallTxAsync` proves, balances, and submits — but returns the transaction ID immediately without waiting for on-chain confirmation. The caller must manually watch for finalization via `publicDataProvider.watchForTxData()`.

This is useful for non-blocking UIs (show "pending" state), custom timeout logic, or parallel transaction submission.

Source — extracted from `low-level-pipeline-e2e.mjs`:

```
// submitCallTxAsync: proves, balances, and submits – but returns
// the txId immediately without waiting for on-chain confirmation.
console.log(' Submitting asynchronously...');
const asyncCallOptions = createCallTxOptions(
  compiledContract, 'increment', contractAddress, privateStateId, [],
);
const submitted = await submitCallTxAsync(providers, asyncCallOptions);

// submitted.txId is available immediately
console.log(` Transaction submitted: ${submitted.txId.slice(0, 16)}...`);

// Now manually watch for finalization via the indexer
console.log(' Waiting for on-chain confirmation...');
const txData = await publicDataProvider.watchForTxData(submitted.txId);
```

[deterministic]

| Step                     | Detail                                        | Verdict |
|--------------------------|-----------------------------------------------|---------|
| Async submission: 2 -> 3 | tx: 002be2bdbcbdb8869... , block: 5063, 17.3s | PASS    |

Verification

[AI-generated]

This example demonstrates three ways to submit a circuit call:

- **Manual pipeline** — `createUnprovenCallTx` → `submitTx`: full control over each stage
- **Rebuilt interface** — `createCircuitCallTxInterface`: reconstruct the high-level API from primitives
- **Async submission** — `submitCallTxAsync` + `watchForTxData`: non-blocking submission with manual confirmation

Each method produces the same result (counter incremented by 1), proving they are functionally equivalent. The low-level APIs compose to match the high-level convenience functions.

The complete source is at `workspace/examples/low-level-pipeline-e2e.mjs`.

### Example 5: Contract Maintenance — Authority and Verifier Keys

Production DApps need to manage their on-chain configuration after deployment: rotating the maintenance authority (who can upgrade the contract), and enabling/disabling individual circuits by inserting or removing their verifier keys. This example demonstrates the full maintenance lifecycle using the counter contract.

Source: `workspace/examples/contract-maintenance-e2e.mjs`

#### The contract

This example reuses the counter contract from Example 1 — the focus is on the maintenance APIs, not the contract logic.

Step 1: Import maintenance APIs

[AI-generated]

The SDK provides both low-level functions and high-level interfaces for contract maintenance. The low-level functions (`submitReplaceAuthorityTx`, `submitRemoveVerifierKeyTx`, `submitInsertVerifierKeyTx`) are curried for partial application. The high-level interfaces (`createContractMaintenanceTxInterface`,

`createCircuitMaintenanceTxInterface`, `createCircuitMaintenanceTxInterfaces`) wrap them into ergonomic objects.

**Source** — extracted from `contract-maintenance-e2e.mjs`:

```
const {
  deployContract,
  findDeployedContract,
  createContractMaintenanceTxInterface,
  createCircuitMaintenanceTxInterface,
  createCircuitMaintenanceTxInterfaces,
  submitReplaceAuthorityTx,
  submitRemoveVerifierKeyTx,
  submitInsertVerifierKeyTx,
} = await wsImport('@midnight-ntwrk/midnight-js-contracts/dist/index.mjs');
```

## Step 2: Replace contract authority [AI-generated]

Every deployed contract has a maintenance authority — the signing key that controls on-chain configuration changes.

`submitReplaceAuthorityTx` transfers this authority to a new key. After replacement, only the new key can manage verifier keys or transfer authority again. Importantly, replacing the authority does **not** affect circuit operation — the contract continues to work normally.

**Source** — extracted from `contract-maintenance-e2e.mjs`:

```
// Generate a new authority signing key
const newAuthority = sampleSigningKey();

// Replace the contract's maintenance authority.
// After this, only the new key can manage verifier keys.
const replaceResult = await submitReplaceAuthorityTx(
  providers,
  compiledContract,
  contractAddress,
)(newAuthority);
```

## Step 3: Create maintenance interfaces [AI-generated]

The SDK provides two levels of maintenance interfaces:

- **Contract level** (`createContractMaintenanceTxInterface`) — authority management via `replaceAuthority(newKey)`
- **Circuit level** (`createCircuitMaintenanceTxInterface`) — per-circuit verifier key management via `removeVerifierKey()` and `insertVerifierKey(newKey)`
- **Bulk** (`createCircuitMaintenanceTxInterfaces`) — creates circuit-level interfaces for all circuits at once, keyed by circuit name

**Source** — extracted from `contract-maintenance-e2e.mjs`:

```
// Contract-level maintenance interface (authority management)
const contractMaintenance = createContractMaintenanceTxInterface(
  providers,
  compiledContract,
  contractAddress,
);

// Circuit-level maintenance interfaces for all circuits
const circuitInterfaces = createCircuitMaintenanceTxInterfaces(
  providers,
  compiledContract,
  contractAddress,
);

// Or create an interface for a single circuit:
const incrementMaintenance = createCircuitMaintenanceTxInterface(
  providers,
  'increment',
  compiledContract,
  contractAddress,
);
```

[deterministic]

| Step                           | Detail | Verdict |
|--------------------------------|--------|---------|
| Maintenance interfaces created |        | PASS    |

Step 4: Disable a circuit by removing its verifier key [AI-generated]

Removing a verifier key from the blockchain makes it impossible to verify proofs for that circuit. Any transaction invoking the circuit will be rejected by the node. This is useful for planned maintenance windows: disable a circuit, deploy updated keys, then re-enable.

Source — extracted from contract-maintenance-e2e.mjs :

```
const removeResult = await incrementMaintenance.removeVerifierKey();
```

Step 5: Re-enable a circuit by inserting a verifier key [AI-generated]

Re-inserting a verifier key re-enables the circuit. In production, you might insert a **new** key (after upgrading the circuit logic). Here we re-insert the same key to confirm the re-enable path works.

Source — extracted from contract-maintenance-e2e.mjs :

```
const insertResult = await incrementMaintenance.insertVerifierKey(verifierKey);
```

## Step 6: High-level authority interface [AI-generated]

The `contractMaintenance.replaceAuthority()` method wraps `submitReplaceAuthorityTx` into a cleaner API — both achieve the same result.

**Source** — extracted from `contract-maintenance-e2e.mjs`:

```
const anotherAuthority = sampleSigningKey();
const replaceResult2 = await contractMaintenance.replaceAuthority(anotherAuthority);
```

## Verification [AI-generated]

This example demonstrates the complete maintenance lifecycle:

- **Authority transfer** — `submitReplaceAuthorityTx` and `contractMaintenance.replaceAuthority()` both transfer maintenance control to a new signing key
- **Circuit disabling** — removing the verifier key prevents the circuit from being called; the counter does not change
- **Circuit re-enabling** — inserting the verifier key restores circuit operation; the counter increments normally
- **Independence** — authority changes do not affect circuit operation; all increments succeed regardless of which key holds authority

The complete source is at `workspace/examples/contract-maintenance-e2e.mjs`.

---

## Example 6: Private State — Encryption, Backup, and Migration

**Tier:** Standalone (no devnet required)

**What it demonstrates:**

- `StorageEncryption`: AES-256-GCM encrypt/decrypt with PBKDF2 key derivation
- Password rotation on private state and signing key stores
- Export/import for backup and restore of private states and signing keys
- Account-scoped migration for legacy unscoped stores
- `decryptValue`: smart decryption handling encrypted and plaintext data
- `DEFAULT_CONFIG`: default store name constants

**APIs exercised:**

- `StorageEncryption` (class) — constructor, encrypt, decrypt, static `isEncrypted`, static `getVersion`
- `decryptValue` (function) — smart decryption for V1/V2/unencrypted data
- `migrateToAccountScoped` (function) — unscoped → account-scoped storage
- `DEFAULT_CONFIG` (constant) — default database store names
- `levelPrivatestateProvider` — factory returning provider + `changePassword`
  - `changeSigningKeysPassword` + `invalidateEncryptionCache` + `exportPrivateStates`
  - `importPrivateStates` + `exportSigningKeys` + `importSigningKeys`

## Step 1: Load module and examine `DEFAULT_CONFIG` [AI-generated]

The private state provider module exports five key items beyond the factory function itself.

**Source** — extracted from `private-state-e2e.mjs`:

```
const {
  levelPrivateStateProvider,
  StorageEncryption,
  decryptValue,
  migrateToAccountScoped,
  DEFAULT_CONFIG,
} = await wsImport(
  '@midnight-ntwrk/midnight-js-level-private-state-provider/dist/index.mjs'
);
```

```
const hasDefaultConfig = DEFAULT_CONFIG != null;
const configKeys = hasDefaultConfig ? Object.keys(DEFAULT_CONFIG) : [];
```

## Step 2: StorageEncryption encrypt/decrypt roundtrip [AI-generated]

StorageEncryption wraps AES-256-GCM encryption with PBKDF2 key derivation. Encrypting and decrypting should produce the original value, and the ciphertext must differ from the plaintext.

**Source** — extracted from `private-state-e2e.mjs`:

```
const password = 'ExamplePassword1!';
const encryption = new StorageEncryption(password);

// Encrypt a test value
const testValue = Buffer.from('hello midnight private state');
const encrypted = await encryption.encrypt(testValue);

// Verify the encrypted value differs from plaintext
// encrypt() returns a string (base64) or Uint8Array depending on version
const encryptedBuf = typeof encrypted === 'string'
  ? Buffer.from(encrypted, 'base64')
  : Buffer.from(encrypted);
const encryptedDiffersFromPlaintext =
  Buffer.compare(encryptedBuf, testValue) !== 0;

// Decrypt and verify roundtrip
const decrypted = await encryption.decrypt(encrypted);
const decryptedBuf = typeof decrypted === 'string'
  ? Buffer.from(decrypted)
  : Buffer.from(decrypted);
const roundtripMatches = Buffer.compare(decryptedBuf, testValue) === 0;
```

## Step 3: Wrong password detection [AI-generated]

Attempting to decrypt with the wrong password must throw an error — this confirms the encryption is real and not just encoding.

**Source** — extracted from `private-state-e2e.mjs`:

```
const wrongEncryption = new StorageEncryption('WrongPassword!');
let wrongPasswordFailed = false;
let wrongPasswordError = '';
try {
  await wrongEncryption.decrypt(encrypted);
  // If we get here, decryption didn't throw – unexpected
  wrongPasswordFailed = false;
  wrongPasswordError = 'Decryption with wrong password did not throw';
} catch (err) {
  wrongPasswordFailed = true;
  wrongPasswordError = err.message || String(err);
}
```

[deterministic]

| Step                            | Detail | Verdict |
|---------------------------------|--------|---------|
| Wrong password fails to decrypt |        | PASS    |

Step 4: Static detection methods [AI-generated]

`StorageEncryption.isEncrypted()` detects encrypted data, and `StorageEncryption.getVersion()` returns the encryption version. Plaintext data is correctly identified as unencrypted.

Source — extracted from `private-state-e2e.mjs`:

```
// isEncrypted/getVersion may expect the raw encrypted form
const isEncryptedResult = StorageEncryption.isEncrypted(encrypted);
const isEncryptedPlain = StorageEncryption.isEncrypted(
  Buffer.from('plain text data'),
);
const encryptionVersion = StorageEncryption.getVersion(encrypted);
```

Step 5: Smart decryption with decryptValue [AI-generated]

`decryptValue` handles both encrypted and plaintext data transparently. This is useful during migration from unencrypted to encrypted storage.

Source — extracted from `private-state-e2e.mjs`:

```
// decryptValue handles encrypted, v1, v2, and unencrypted data
const decryptedSmart = await decryptValue(encrypted, encryption);
const decryptedSmartBuf = typeof decryptedSmart === 'string'
  ? Buffer.from(decryptedSmart)
  : Buffer.from(decryptedSmart);
const smartDecryptMatches = Buffer.compare(decryptedSmartBuf, testValue) === 0;

// It should also handle plaintext gracefully (pass-through)
let passthroughMatches = false;
try {
  const plaintextPassthrough = await decryptValue(testValue, encryption);
  const ptBuf = typeof plaintextPassthrough === 'string'
    ? Buffer.from(plaintextPassthrough)
    : Buffer.from(plaintextPassthrough);
  passthroughMatches = Buffer.compare(ptBuf, testValue) === 0;
} catch {
  // Some implementations may not support plaintext passthrough
  passthroughMatches = true; // not a failure – just not supported
}
```

## Step 6: Provider store/retrieve and password rotation [AI-generated]

After storing private state and signing keys, rotating the password re-encrypts all data. A new provider with the new password can still access the data. `invalidateEncryptionCache()` forces re-derivation of the encryption key.

**Source** — extracted from `private-state-e2e.mjs`:

```
const provider = levelPrivateStateProvider({
  privateStateStoreName: join(storeDir, 'private-state'),
  signingKeyStoreName: join(storeDir, 'signing-keys'),
  privateStoragePasswordProvider: () => password,
  accountId,
});

// Set a contract address before using private state get/set.
// In production, this is done automatically by the contracts API.
const fakeContractAddress = '0'.repeat(64);
provider.setContractAddress(fakeContractAddress);
```



```
const newPassword = 'NewSecurePassword2!';

// changePassword(oldPasswordProvider, newPasswordProvider, options?)
// Both arguments are functions that return the password string.
const rotateResult = await provider.changePassword(
  () => password,
  () => newPassword,
);

// changeSigningKeysPassword has the same signature
const rotateSigningResult = await provider.changeSigningKeysPassword(
  () => password,
  () => newPassword,
);
```

```
// invalidateEncryptionCache forces re-derivation of the encryption key
// on the next operation. This is needed after password rotation when
// using the same provider instance.
provider2.invalidateEncryptionCache();

// Verify the provider still works after cache invalidation
const afterInvalidation = await provider2.get(stateKey);
const cacheInvalidationWorks = afterInvalidation != null &&
  Buffer.compare(Buffer.from(afterInvalidation), Buffer.from(stateValue)) === 0;
```

[deterministic]

| Step                             | Detail | Verdict |
|----------------------------------|--------|---------|
| Password rotation preserves data |        | PASS    |

Step 7: Export/import for backup and restore [AI-generated]

Private states and signing keys can be exported with a separate export password. The exported data can be imported into a completely fresh provider, recreating the original encrypted stores.

Source — extracted from `private-state-e2e.mjs`:

```
// exportPrivateStates returns an object with format, encryptedPayload, salt
const exported = await provider2.exportPrivateStates({
  password: newPassword,
});

const hasExportData = exported != null;
const exportType = typeof exported;
const exportKeys = hasExportData ? Object.keys(exported) : [];
```

```
const importProvider = levelPrivateStateProvider({
  privateStateStoreName: join(importDir, 'private-state'),
  signingKeyStoreName: join(importDir, 'signing-keys'),
  privateStoragePasswordProvider: () => newPassword,
  accountId,
});
importProvider.setContractAddress(fakeContractAddress);

// importPrivateStates(exportData, options?) – takes the export object directly
const importResult = await importProvider.importPrivateStates(exported);
```

```
const exportedKeys = await provider2.exportSigningKeys({
  password: newPassword,
});
```

Step 8: Account-scoped migration

[AI-generated]

Legacy private state stores without account scoping can be migrated to account-scoped storage using `migrateToAccountScoped`. This adds account isolation by hashing the account ID into the storage key path.

Source — extracted from `private-state-e2e.mjs`:

```
// migrateToAccountScoped(config) copies data from the base store
// to an account-scoped store, keyed by a hashed accountId.
const targetAccountId = 'migrated-account-001';
let migrationResult;
let migrationWorked = false;
try {
  migrationResult = await migrateToAccountScoped({
    privateStateStoreName: legacyStoreName,
    signingKeyStoreName: legacySKStoreName,
    privateStoragePasswordProvider: () => password,
    accountId: targetAccountId,
  });
  migrationWorked = true;
} catch (err) {
  migrationResult = { error: err.message || String(err) };
  migrationWorked = false;
}
```

[deterministic]

| Step                     | Detail | Verdict |
|--------------------------|--------|---------|
| Account-scoped migration |        | PASS    |

Verification

[AI-generated]

This example demonstrates the complete private state management lifecycle:

- **Encryption** — `StorageEncryption` provides AES-256-GCM with PBKDF2 key derivation; wrong passwords are rejected

- **Static detection** — `isEncrypted` and `getVersion` identify encrypted data
- **Smart decryption** — `decryptValue` handles encrypted and plaintext transparently
- **Password rotation** — `changePassword` and `changeSigningKeysPassword` re-encrypt all stored data without data loss
- **Export/import** — private states and signing keys can be backed up and restored with a separate export password
- **Migration** — `migrateToAccountScoped` adds account isolation to legacy unscoped stores
- **Cache management** — `invalidateEncryptionCache` forces re-derivation
- **Cleanup** — `remove` and `removeSigningKey` delete individual entries

The complete source is at `workspace/examples/private-state-e2e.mjs`.

## Example 7: Utility Functions, Type Helpers, and Network Configuration

**Standalone** — no devnet required. Exercises the `utils`, `types`, and `network-id` packages.

**Evidence file:** `docs/utils-types-evidence.json` **Source:** `workspace/examples/utils-types-e2e.mjs`

### Step 1: Load modules [deterministic]

```
requires-claim: modulesLoaded
```

```
// [extract: utils-imports from workspace/examples/utils-types-e2e.mjs]
```

```
// [extract: types-imports from workspace/examples/utils-types-e2e.mjs]
```

```
// [extract: networkid-imports from workspace/examples/utils-types-e2e.mjs]
```

Loads utility functions, type helpers, error classes, and network-id management from three SDK packages.

### Step 2: Hex encoding/decoding roundtrip [deterministic]

```
requires-claim: hexRoundtrip
```

```
// [extract: hex-roundtrip from workspace/examples/utils-types-e2e.mjs]
```

`toHex` encodes a `Uint8Array` to a hex string; `fromHex` decodes it back. The roundtrip preserves byte content exactly.

### Step 3: isHex and parseHex [deterministic]

```
requires-claim: validHexAccepted, invalidCharsRejected, wrongLengthRejected
```

```
// [extract: hex-validation from workspace/examples/utils-types-e2e.mjs]
```

`isHex` validates hex strings with optional byte-length checking. Both `0x`-prefixed and bare hex are accepted. `parseHex` returns a structured result with `hasPrefix`, `byteChars`, and `incompleteChars` fields.

#### Step 4: `assertIsHex` and `assertIsContractAddress` [deterministic]

requires-claim: `assertIsHexThrowsInvalid`, `assertIsContractAddressPrefixRejected`

```
// [extract: hex-assertions from workspace/examples/utis-types-e2e.mjs]
```

`assertIsHex` throws on invalid hex. `assertIsContractAddress` requires exactly 32 bytes (64 hex chars) with **no** `0x` prefix — the prefix itself causes rejection.

#### Step 5: `assertDefined` and `assertUndefined` [deterministic]

requires-claim: `assertDefinedRejectsNull`, `assertDefinedRejectsZero`

```
// [extract: assert-defined from workspace/examples/utis-types-e2e.mjs]
```

`assertDefined` is a **truthiness** check — it rejects not just `null` and `undefined` but also falsy values like `0`, `false`, and `""`. This is stricter than the TypeScript type signature suggests. `assertUndefined` succeeds only on `null` and `undefined`.

#### Step 6: Branded type constructors [deterministic]

requires-claim: `proverKeyIsUint8Array`, `verifierKeyIsUint8Array`, `zkirIsUint8Array`

```
// [extract: branded-types from workspace/examples/utis-types-e2e.mjs]
```

`createProverKey`, `createVerifierKey`, and `createZKIR` wrap `Uint8Array` data into branded types for type-level safety. At runtime they are still `Uint8Array` instances with the original data preserved.

#### Step 7: `asContractAddress` [deterministic]

requires-claim: `contractAddressCreated`

```
// [extract: as-contract-address from workspace/examples/utis-types-e2e.mjs]
```

`asContractAddress` creates a branded `ContractAddress` value from a hex string.

#### Step 8: Transaction status constants [deterministic]

requires-claim: `allStrings`, `allDistinct`

```
// [extract: tx-status-constants from workspace/examples/utis-types-e2e.mjs]
```

Five string constants for transaction result handling: `SegmentFail`, `SegmentSuccess`, `FailEntirely`, `FailFallible`, `SucceedEntirely`. All are distinct string values used as discriminated union tags.

### Step 9: Export limit constants [deterministic]

```
requires-claim: bothPositive
```

```
// [extract: export-limits from workspace/examples/utls-types-e2e.mjs]
```

`MAX_EXPORT_STATES` and `MAX_EXPORT_SIGNING_KEYS` are numeric constants that cap the maximum number of items that can be exported/imported in a single operation, preventing memory exhaustion.

### Step 10: Error class hierarchy [deterministic]

```
requires-claim: exportDecryptionExtendsImport, allExtendError
```

```
// [extract: error-classes from workspace/examples/utls-types-e2e.mjs]
```

Seven error classes form a hierarchy:

- `PrivateStateImportError` (base) — generic import failure
  - `ExportDecryptionError` — decryption failed (no details, preventing oracle attacks)
  - `InvalidExportFormatError` — export data format invalid
  - `ImportConflictError` — import conflicts with existing data (has `conflictCount`)
- `PrivateStateExportError` — export failure
- `SigningKeyExportError` — signing key export failure
- `InvalidProtocolSchemeError` — invalid protocol in URL

All extend `Error`. The three import-specific subclasses extend `PrivateStateImportError`.

### Step 11: Network ID [deterministic]

```
requires-claim: setAndGet, canUpdate
```

```
// [extract: network-id from workspace/examples/utls-types-e2e.mjs]
```

`setNetworkId` / `getNetworkId` manage a **global module-level** network identifier. The ID must be set before any wallet or contract operation. It can be updated at any time.

### Step 12: Key parsing [deterministic]

```
requires-claim: coinPublicKeyHexPassthrough, encPublicKeyHexPassthrough
```

```
// [extract: key-parsing from workspace/examples/utls-types-e2e.mjs]
```

`parseCoinPublicKeyToHex` and `parseEncPublicKeyToHex` accept either Bech32m-encoded or hex-encoded public keys and normalize to hex. When given a plain hex string, they pass it through unchanged.

### Step 13: `ttlOneHour` [deterministic]

requires-claim: returnsDate, approximatelyOneHour

```
// [extract: ttl-one-hour from workspace/examples/utls-types-e2e.mjs]
```

`ttlOneHour` returns a `Date` object set to one hour from now — a convenience for setting transaction time-to-live.

### Step 14: `LogLevel` enum [deterministic]

requires-claim: hasSixLevels, valuesAreStrings

```
// [extract: log-level from workspace/examples/utls-types-e2e.mjs]
```

`LogLevel` defines six standard log severity levels (INFO, WARN, ERROR, FATAL, DEBUG, TRACE) used by `LoggerProvider`. Each key maps to a lowercase string value.

## Verification [deterministic]

| Claim                                              | Evidence                                                                                         |
|----------------------------------------------------|--------------------------------------------------------------------------------------------------|
| <code>modulesLoaded</code>                         | All three packages loaded successfully                                                           |
| <code>hexRoundtrip</code>                          | <code>toHex(fromHex(hex)) === hex</code>                                                         |
| <code>validHexAccepted</code>                      | <code>isHex('deadbeef')</code> returns true                                                      |
| <code>invalidCharsRejected</code>                  | <code>isHex('xyz123')</code> returns false                                                       |
| <code>wrongLengthRejected</code>                   | <code>isHex('deadbeef', 3)</code> returns false                                                  |
| <code>assertIsHexThrowsInvalid</code>              | <code>assertIsHex('xyz')</code> throws                                                           |
| <code>assertIsContractAddressPrefixRejected</code> | <code>0x</code> -prefixed address rejected                                                       |
| <code>assertDefinedRejectsNull</code>              | <code>assertDefined(null)</code> throws                                                          |
| <code>assertDefinedRejectsZero</code>              | <code>assertDefined(0)</code> throws (truthiness check)                                          |
| <code>proverKeyIsUint8Array</code>                 | Branded ProverKey is <code>instanceof Uint8Array</code>                                          |
| <code>verifierKeyIsUint8Array</code>               | Branded VerifierKey is <code>instanceof Uint8Array</code>                                        |
| <code>zkirIsUint8Array</code>                      | Branded ZKIR is <code>instanceof Uint8Array</code>                                               |
| <code>contractAddressCreated</code>                | <code>asContractAddress</code> returns non-null branded type                                     |
| <code>allStrings</code>                            | All 5 status constants are strings                                                               |
| <code>allDistinct</code>                           | All 5 status constants are unique                                                                |
| <code>bothPositive</code>                          | Both export limits are positive numbers                                                          |
| <code>exportDecryptionExtendsImport</code>         | <code>ExportDecryptionError</code> extends <code>PrivateStateImportError</code>                  |
| <code>allExtendError</code>                        | All 7 error classes extend <code>Error</code>                                                    |
| <code>setAndGet</code>                             | <code>setNetworkId('testnet');</code> <code>getNetworkId()</code> returns <code>'testnet'</code> |
| <code>canUpdate</code>                             | Setting a new ID replaces the previous one                                                       |
| <code>coinPublicKeyHexPassthrough</code>           | Hex input passes through unchanged                                                               |
| <code>encPublicKeyHexPassthrough</code>            | Hex input passes through unchanged                                                               |
| <code>returnsDate</code>                           | <code>ttlOneHour()</code> returns a <code>Date</code> instance                                   |
| <code>approximatelyOneHour</code>                  | TTL is approximately one hour from now                                                           |
| <code>hasSixLevels</code>                          | All 6 <code>LogLevel</code> keys exist with string values                                        |
| <code>valuesAreStrings</code>                      | Values are lowercase: info, warn, error, fatal, debug, trace                                     |

## What this example demonstrates [AI-generated]

- **Hex utilities** — encode, decode, validate, and parse hex strings with prefix awareness and byte-length checking
- **Assertion helpers** — `assertDefined` is a truthiness check (rejects 0 and false), not just a null/undefined check
- **Branded types** — `ProverKey`, `VerifierKey`, and `ZKIR` wrap `Uint8Array` with type-level distinction while preserving runtime identity
- **Contract addresses** — validated (32 bytes, no prefix) and branded
- **Transaction status** — five distinct string constants for discriminated union pattern matching

- **Error hierarchy** — structured error classes for private state import/export operations, with `ExportDecryptionError` intentionally hiding failure details to prevent oracle attacks
- **Network configuration** — global module-level network ID required before any SDK operation
- **Key parsing** — polymorphic parsers accepting both Bech32m and hex formats
- **TTL** — convenience helper for transaction time-to-live
- **Log levels** — `LogLevel` enum defines six severity levels (INFO through TRACE) used by `LoggerProvider`

The complete source is at `workspace/examples/utls-types-e2e.mjs`.

## Example 8: Advanced Deploy Pipeline and Provider Configuration [deterministic]

Evidence: `docs/advanced-deploy-evidence.json`

Examples 1–4 used `deployContract` (high-level) and `createUnprovenCallTx` (mid-level) for the deploy/call lifecycle. Example 8 exercises the **remaining SDK APIs** — alternative deploy paths, scoped transactions, provider configuration, and maintenance error classes — covering 22 callable exports that are not reached by the earlier examples.

### Step 1: `submitDeployTx` — alternative high-level deploy

`submitDeployTx` deploys a contract but returns raw transaction data instead of a `CallableContractAPI`. This is useful when callers need to manage the contract interface themselves.

```
const deployResult = await submitDeployTx(providers, deployOptions);
```

**Evidence:** The deploy succeeds and produces a valid `contractAddress` (64-char hex string) and `initialContractState`.

### Step 2: `createUnprovenDeployTx` — mid-level deploy

`createUnprovenDeployTx` creates an unproven deploy transaction without submitting it. This allows inspection or modification before submission.

```
const unprovenDeploy = await createUnprovenDeployTx(providers, {
  compiledContract, initialPrivateState: {},
});
```

**Evidence:** Returns an object with `private.unprovenTx` (the unsigned transaction data).

### Step 3: `createUnprovenDeployTxFromVerifierKeys` — lowest-level deploy

`createUnprovenDeployTxFromVerifierKeys` takes individual arguments (`zkConfigProvider`, `coinPublicKey`, `options`, `encryptionPublicKey`) rather than the full providers bundle. This is the function that `createUnprovenDeployTx` delegates to internally.

```
const vkResult = await createUnprovenDeployTxFromVerifierKeys(
  zkConfigProvider, coinPubKeyHex,
  { compiledContract, initialPrivateState: {} },
  walletProvider.getEncryptionPublicKey(),
);
```



**Evidence:** Returns an object with `private.unprovenTx`.

### Steps 4–5: Deploy and call — baseline via `deployContract`

Standard deploy via `deployContract` (same as Examples 1–4), followed by a `submitCallTx` to increment the counter. This establishes the contract state used by subsequent steps.

**Evidence:** Counter value goes from 0 to 1.

### Step 6: `createUnprovenCallTxFromInitialStates` — call with explicit state

`createUnprovenCallTxFromInitialStates` is the lowest-level call API. Instead of letting the SDK query state, the caller provides explicit initial states fetched via `publicDataProvider.queryZSwapAndContractState(contractAddress)`.

```
const [zswapChainState, contractState] =
  await publicDataProvider.queryZSwapAndContractState(contractAddress);
const isResult = await createUnprovenCallTxFromInitialStates(
  zkConfigProvider,
  { compiledContract, contractAddress, circuitId: 'increment',
    coinPublicKey: coinPubKey, initialContractState: contractState,
    initialZswapChainState: zswapChainState, args: [] },
  walletProvider.getEncryptionPublicKey(),
);
```

**Evidence:** Returns an object with `private.unprovenTx`.

### Step 7: `submitTxAsync` — non-blocking tx submit

`submitTxAsync` submits a transaction and returns the `txId` immediately without waiting for proof generation or finalization. The caller can then poll for status using `watchForTxData`.

```
const unprovenCallData = await createUnprovenCallTx(providers, callOptions);
const txId = await submitTxAsync(providers, {
  unprovenTx: unprovenCallData.private.unprovenTx,
  circuitId: 'increment',
});
// txId is a hex string; poll via watchForTxData
```

**Evidence:** Returns a transaction ID (hex string). Counter value increments from 1 to 2 after finalization.

### Step 8: `withContractScopedTransaction` — batched scoped calls

`withContractScopedTransaction` groups one or more call transactions into a scoped context, ensuring they execute atomically against the same contract state snapshot.

```
await withContractScopedTransaction(providers, async (txCtx) => {
  await submitCallTx(providers, scopedCallOptions, txCtx);
});
```

**Evidence:** Counter value increments from 2 to 3.

## Steps 9–22: Standalone verifications

The remaining steps verify the existence and correct types of exports that don't require devnet interaction:

| Steps | What they verify                                                                                                                                                                                                  |
|-------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 9–11  | Maintenance error classes ( <code>InsertVerifierKeyTxFailedError</code> , <code>RemoveVerifierKeyTxFailedError</code> , <code>ReplaceMaintenanceAuthorityTxFailedError</code> ) extend <code>TxFailedError</code> |
| 12–15 | Provider helpers ( <code>FetchZkConfigProvider</code> , <code>DEFAULT_TIMEOUT</code> = 300000ms, <code>httpClientProvingProvider</code> , <code>LoggerProvider</code> )                                           |
| 16–17 | <code>ZKConfigProvider</code> interface and <code>zkConfigToProvingKeyMaterial</code> conversion                                                                                                                  |
| 18–19 | Indexer utilities ( <code>toUnshieldedBalances</code> , <code>toUnshieldedUtxos</code> , <code>IndexerFormattedError</code> )                                                                                     |
| 20–22 | Type helpers ( <code>asEffectOption</code> , <code>makeContractExecutableRuntime</code> , <code>IncompleteCallTxPrivateStateConfig</code> , <code>IncompleteFindContractPrivateStateConfig</code> )               |

### Verification evidence [deterministic]

| Claim                                                                     | Evidence                                                                                       |
|---------------------------------------------------------------------------|------------------------------------------------------------------------------------------------|
| <code>submitDeployTx</code> deploys and returns raw tx data               | <code>advanced-deploy-evidence.json</code> step 1: <code>contractAddress</code> is 64-char hex |
| <code>createUnprovenDeployTx</code> creates unproven deploy               | step 2: returns <code>private.unprovenTx</code>                                                |
| <code>createUnprovenDeployTxFromVerifierKeys</code> takes individual args | step 3: returns <code>private.unprovenTx</code>                                                |
| <code>createUnprovenCallTxFromInitialStates</code> accepts explicit state | step 6: returns <code>private.unprovenTx</code>                                                |
| <code>submitTxAsync</code> returns <code>txId</code> without blocking     | step 7: <code>txId</code> is hex string, counter increments                                    |
| <code>withContractScopedTransaction</code> batches scoped calls           | step 8: counter increments within scope                                                        |
| Maintenance errors extend <code>TxFailedError</code>                      | steps 9–11: <code>instanceof TxFailedError</code> is true                                      |
| <code>DEFAULT_TIMEOUT</code> is 300000ms                                  | step 15: value === 300000                                                                      |
| <code>httpClientProvingProvider</code> is a factory function              | step 14: <code>typeof</code> === 'function'                                                    |
| <code>FetchZkConfigProvider</code> is constructable                       | step 12: <code>typeof</code> === 'function'                                                    |
| <code>LoggerProvider</code> is constructable                              | step 13: <code>typeof</code> === 'function'                                                    |
| Indexer utilities are functions                                           | steps 18–19: <code>typeof</code> === 'function'                                                |
| Type helpers exist with correct types                                     | steps 20–22: <code>typeof</code> checks pass                                                   |

### What this example demonstrates [AI-generated]

- **Alternative deploy paths** — three levels of abstraction: `deployContract` (high-level, returns callable API), `submitDeployTx` (high-level, returns raw data), `createUnprovenDeployTx` / `createUnprovenDeployTxFromVerifierKeys` (mid/low-level, returns unproven tx for manual submission)
- **Explicit state management** — `createUnprovenCallTxFromInitialStates` lets callers provide pre-fetched chain state instead of relying on SDK auto-queries

- **Async transaction submission** — `submitTxAsync` returns immediately with a txId for polling-based workflows
- **Scoped transactions** — `withContractScopedTransaction` ensures atomic execution against a consistent state snapshot
- **Maintenance errors** — typed error hierarchy for verifier key and maintenance authority operations
- **Provider configuration** — `FetchZkConfigProvider` for ZK artifact retrieval, `httpClientProvingProvider` for remote proving, `LoggerProvider` for structured logging
- **Type system** — `asEffectOption` for Option type interop, `makeContractExecutableRuntime` for runtime construction, `IncompleteCallTxPrivateStateConfig` and `IncompleteFindContractPrivateStateConfig` for partial configs

The complete source is at `workspace/examples/advanced-deploy-e2e.mjs`.

---

# Appendix A: Execution Oracle Tool [deterministic]

Description of the `tools/trace-sdk-oracle.mjs` tool.

## Input

A test case JSON file:

```
{
  "name": "counter: increment returns 1",
  "source": "test-contracts/counter.compact",
  "circuit": "increment",
  "witnesses": {},
  "expect": "accept"
}
```

## Pipeline

1. Compile `.compact` source via `compactc`
2. Load compiled `Contract` class via dynamic import
3. Initialize contract state via constructor
4. Execute specified circuit with witnesses
5. Capture proof data and serialize via `proofDataIntoSerializedPreimage`
6. Apply tamper if specified
7. Verify via ZKIR WASM checker
8. Report verdict: ACCEPTED/REJECTED/EXECUTE-ERROR

## Tamper modes

| Target                         | What changes                | Proves                             |
|--------------------------------|-----------------------------|------------------------------------|
| <code>output</code>            | Modify circuit output bytes | Output constraints enforced        |
| <code>privateTranscript</code> | Modify witness bytes        | Private input constraints enforced |
| <code>publicTranscript</code>  | Modify transcript entry     | Ledger interaction integrity       |

## Appendix B: API Surface Extraction Tool [deterministic]

---

Description of the `tools/extract-api-surface.mjs` tool.

### What it extracts

For each package under `upstream/midnight-js/packages/`:

1. Reads the barrel `src/index.ts` file
2. Resolves `export *` to underlying source files
3. Extracts every exported symbol with its kind (type, const, class, interface, function, enum, abstract-class)
4. Produces structured JSON and/or Markdown

### Modes

```
node tools/extract-api-surface.mjs --json      # Structured JSON
node tools/extract-api-surface.mjs --markdown  # Human-readable table
node tools/extract-api-surface.mjs --diff old.json # Change detection
```

---

## Appendix C: Coverage Checker Tool [deterministic]

---

Description of the `tools/check-sdk-coverage.mjs` tool.

Cross-references the API surface against:

1. **Documentation** -- does each export have a section in this outline?
  2. **Test evidence** -- does each testable export have an oracle test?
  3. **Change detection** -- are there new exports not yet documented?
-

## Appendix D: Reproducing This Document

---

```
# Extract the API surface
node tools/extract-api-surface.mjs --json > docs/api-surface.json
node tools/extract-api-surface.mjs --markdown > docs/api-surface.md

# Run all oracle traces
node tools/trace-sdk-oracle.mjs --verbose --cache-dir workspace/.cache/bls-params \
  workspace/test-cases/*.json

# Check documentation coverage
node tools/check-sdk-coverage.mjs --strict
```

---

*Document assembled from 249 SDK tests (249 passed) and 14 typecheck modules (389 type assertions). Generated: 2026-03-13.*